

软件工程概述

软件工程与实践课程组

电子科技大学信息与软件工程学院

软件工程与实践

许毅

电子科技大学 信息与软件工程学院

xuyi0421@uestc.edu.cn

目录

- 程序及质量保证方法
- 软件及其特点
- 软件工程概述

Part1：程序及质量保证方法

1. 程序及质量要求

- ✓程序及其内部和外部质量

2. 程序质量保证方法

- ✓编码规范、设计方法、代码重用、结对编程

3. 程序质量的分析方法

- ✓人工审查、自动化分析、代码测试

4. 编写程序需要解决的问题

- ✓需要工程化的方法支持



1.1 何为程序?

- **程序 (Program)** 是由**程序设计语言**所描述的、能为计算机所理解和处理的一组语句序列
 - ✓ 用程序设计语言 (Programming Language) 来描述的
 - ✓ 如Java、C、C++、Python
- **程序严格遵循程序设计语言的各项**语法和语义**规定**
 - ✓ 确保程序代码能为程序设计语言的编译器所理解，进而编译生成相应的可运行代码
- **程序代码可表现为二种形式**
 - ✓ 源代码 (Source Code)：用程序设计语言所描述的代码
 - ✓ 可执行代码 (Executable Code)：可执行的二进制或中间码

程序代码示例

```
import java.io.*;

public class Main {
    public static void main(String[] args) {
        try {
            BufferedReader in = new BufferedReader(new FileReader("test.log"));
            String str;
            while ((str = in.readLine()) != null) {
                System.out.println(str);
            }
            System.out.println(str);
        } catch (IOException e) {
        }
    }
}
```

用Java语言
编写的代码

1.2 程序组成-语句

□程序中的语句

- ✓ **声明、定义、控制、计算等**
- ✓ 实现特定的功能
- ✓ 用程序设计语言来描述

□计算机可以理解

- ✓ **可编译成二进制代码**
- ✓ 代码在计算机上可执行

```
1  /*
2   * Copyright (c) 2010-2011, The MiCode Open Source Community (www.micode.net)
3   *
4   * Licensed under the Apache License, Version 2.0 (the "License");
5   * you may not use this file except in compliance with the License.
6   * You may obtain a copy of the License at
7   *
8   *     http://www.apache.org/licenses/LICENSE-2.0
9   *
10  * Unless required by applicable law or agreed to in writing, software
11  * distributed under the License is distributed on an "AS IS" BASIS,
12  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13  * See the License for the specific language governing permissions and
14  * limitations under the License.
15  */
16
17 package net.micode.notes.data;
18
19 import android.content.Context;
20
21 public class Contact {
22     private static HashMap<String, String> sContactCache;
23     private static final String TAG = "Contact";
24
25     private static final String CALLER_ID_SELECTION = "PHONE_NUMBERS_EQUAL(" + Phone.NUMBER
26     + ",?) AND " + Data.MIMETYPE + "='" + Phone.CONTENT_ITEM_TYPE + "'"
27     + " AND " + Data.RAW_CONTACT_ID + " IN "
28     + "(SELECT raw_contact_id "
29     + " FROM phone_lookup"
30     + " WHERE min_match = '+')";
31
32     public static String getContact(Context context, String phoneNumber) {
33         if(sContactCache == null) {
34             sContactCache = new HashMap<String, String>();
35         }
36
37         if(sContactCache.containsKey(phoneNumber)) {
38             return sContactCache.get(phoneNumber);
39         }
40     }
41 }
```

程序组成-模块

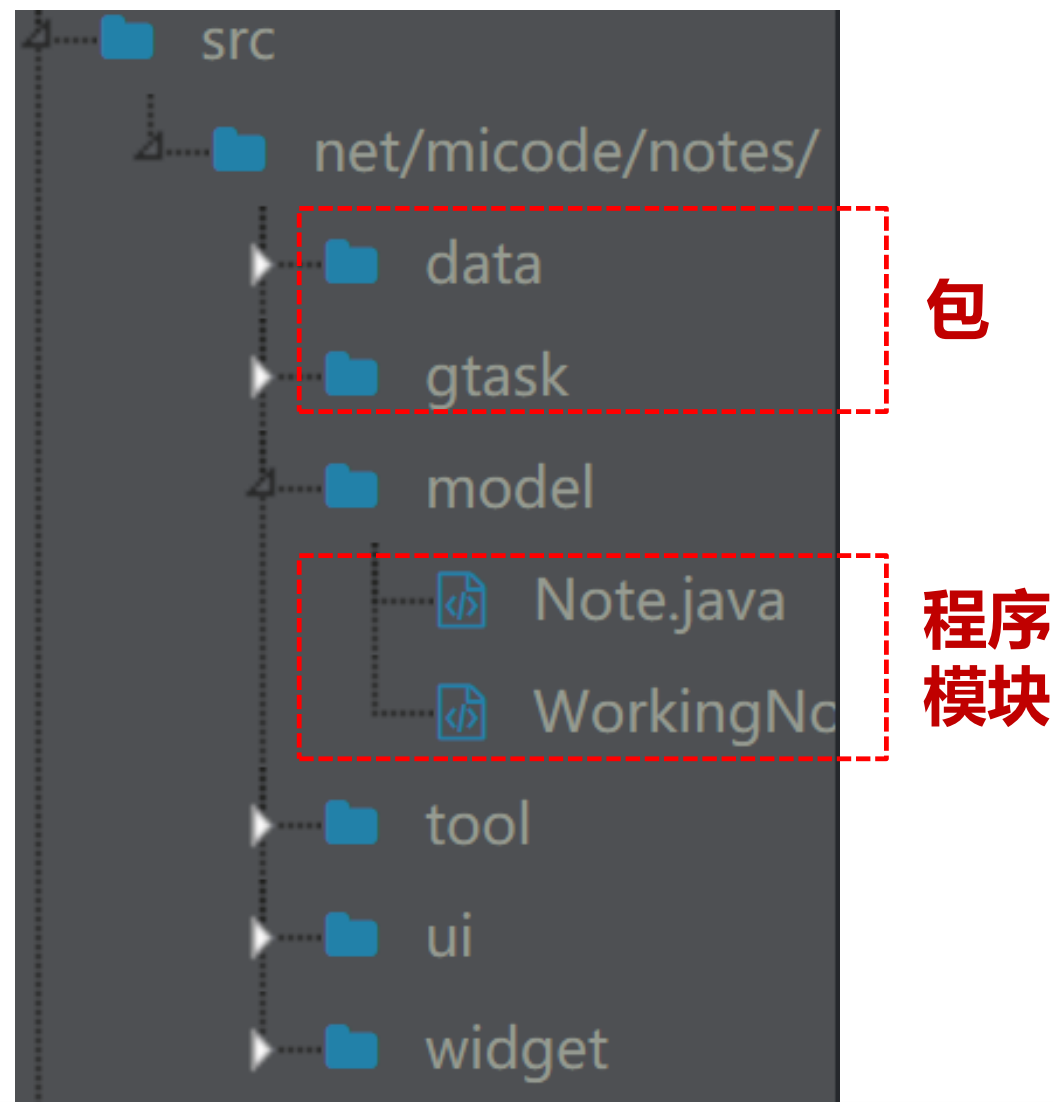
□由诸多相互交互的**模块**组成

- ✓包(Package)
- ✓类(Class)
- ✓方法(Method)

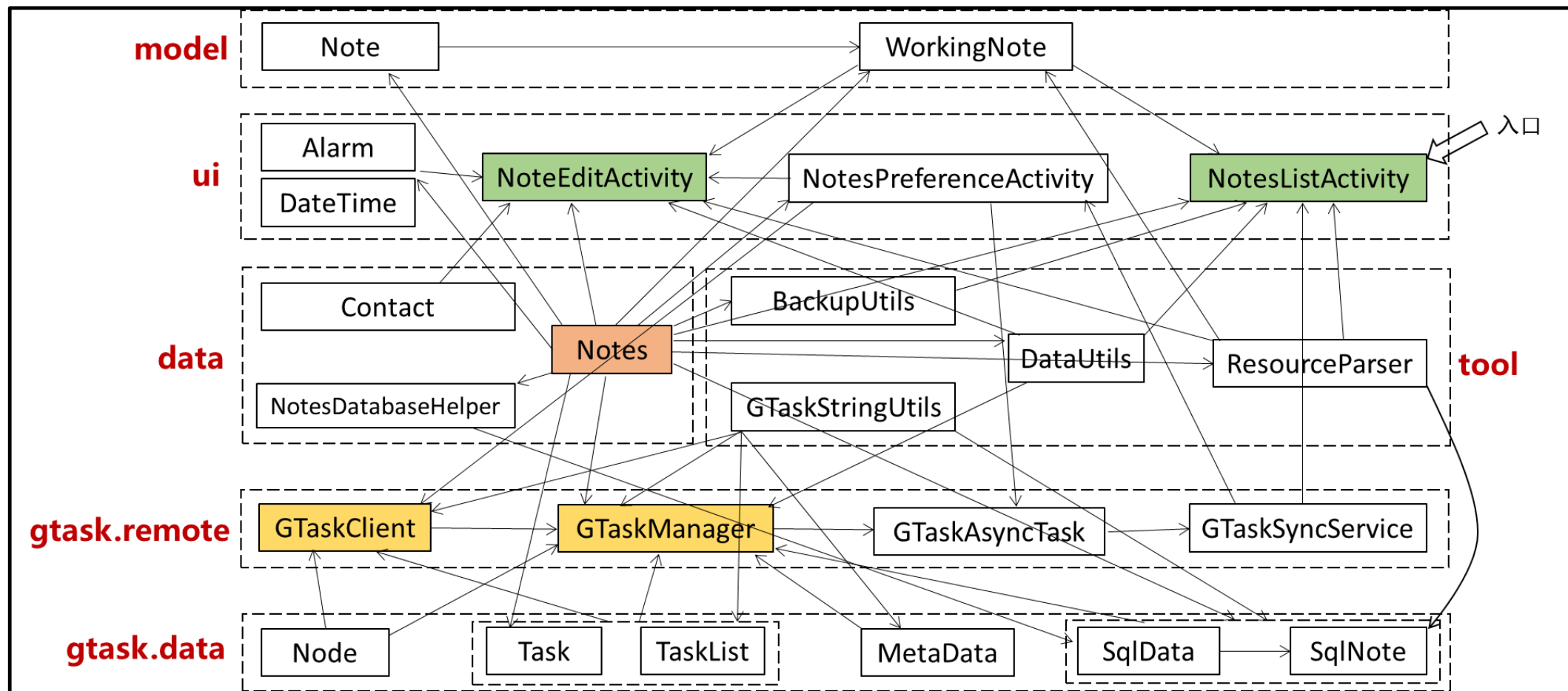
□每个模块实现特定**功能**

□示例：小米便签中的模块

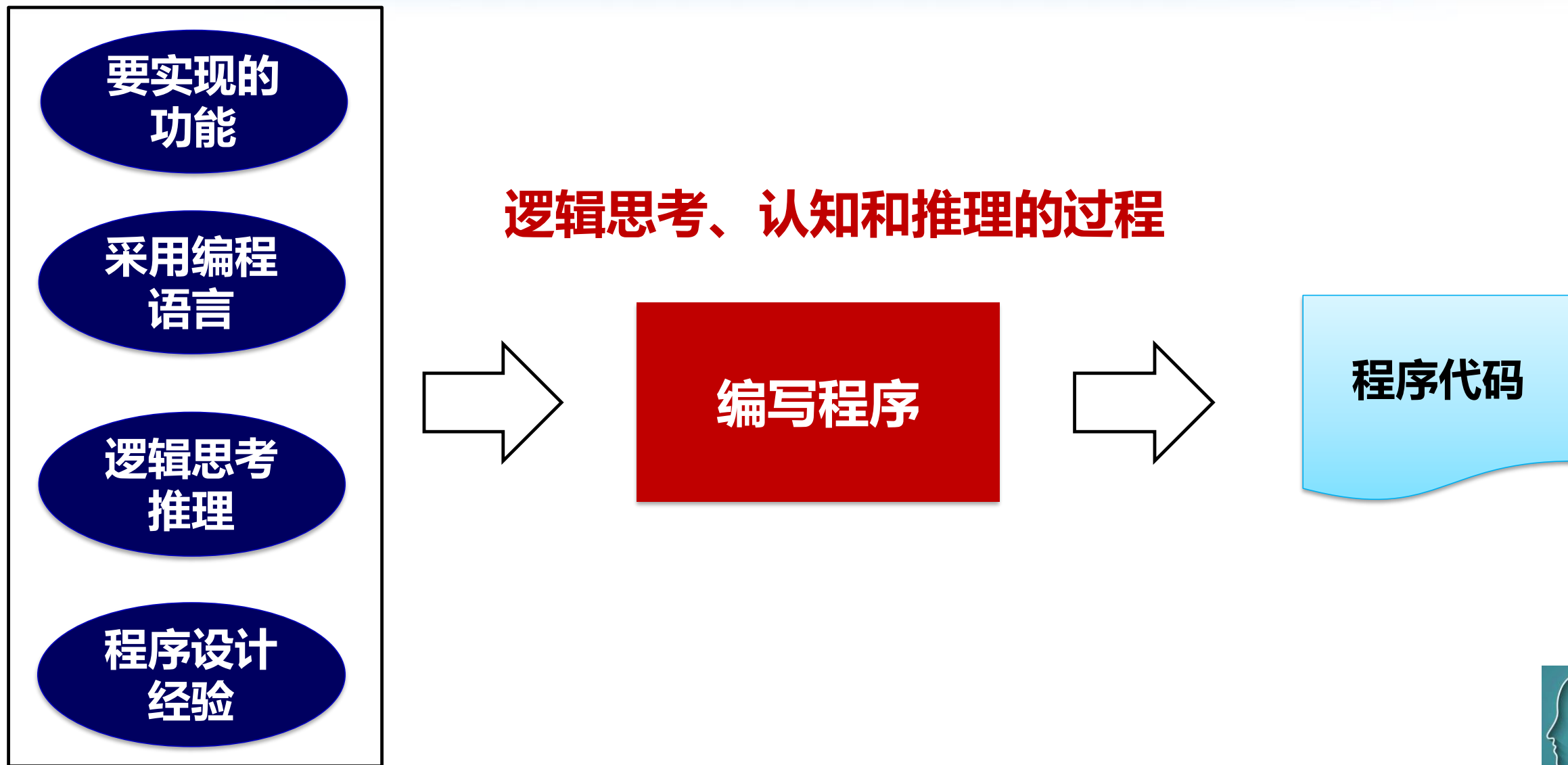
- ✓170个文件、471个方法



示例：小米便签开源软件模块结构



1.3 程序如何编写?



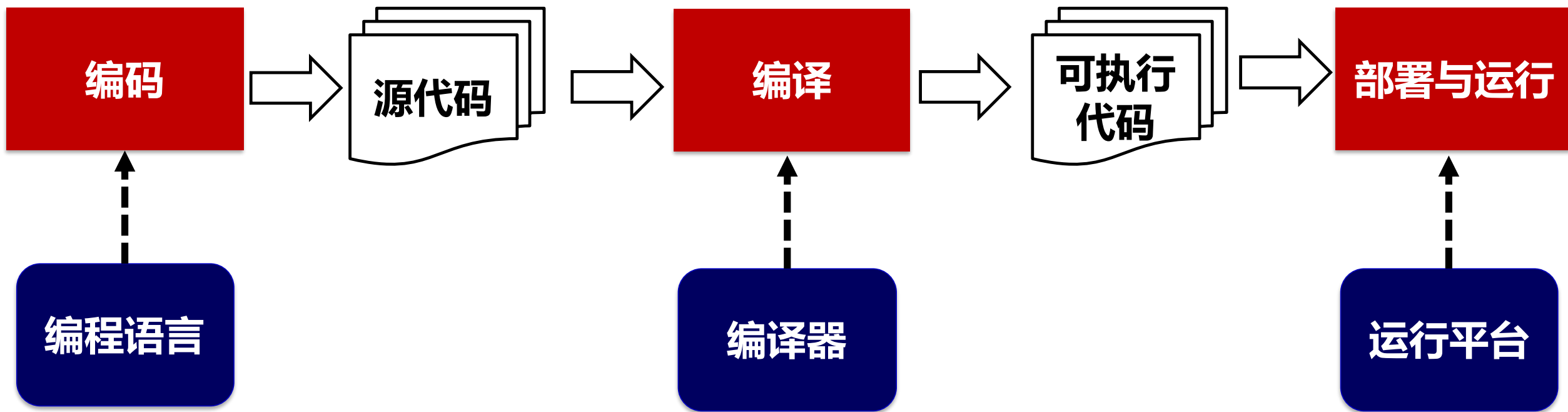
你是如何编写程序的?

程序的编码、编译、部署和运行

如何保证质量问题？！

基于用户需求

产生运行结果



确保程序质量是编码的首要工作！

产品的质量问题

□**外在：**用户直接可以感觉到的

✓**使用用户：**圆滑、手感好、美观、不死机、操作便捷

□**内在：**用户无法直接感触到的

✓**维修人员：**易于维护、易于更换



外在



内在

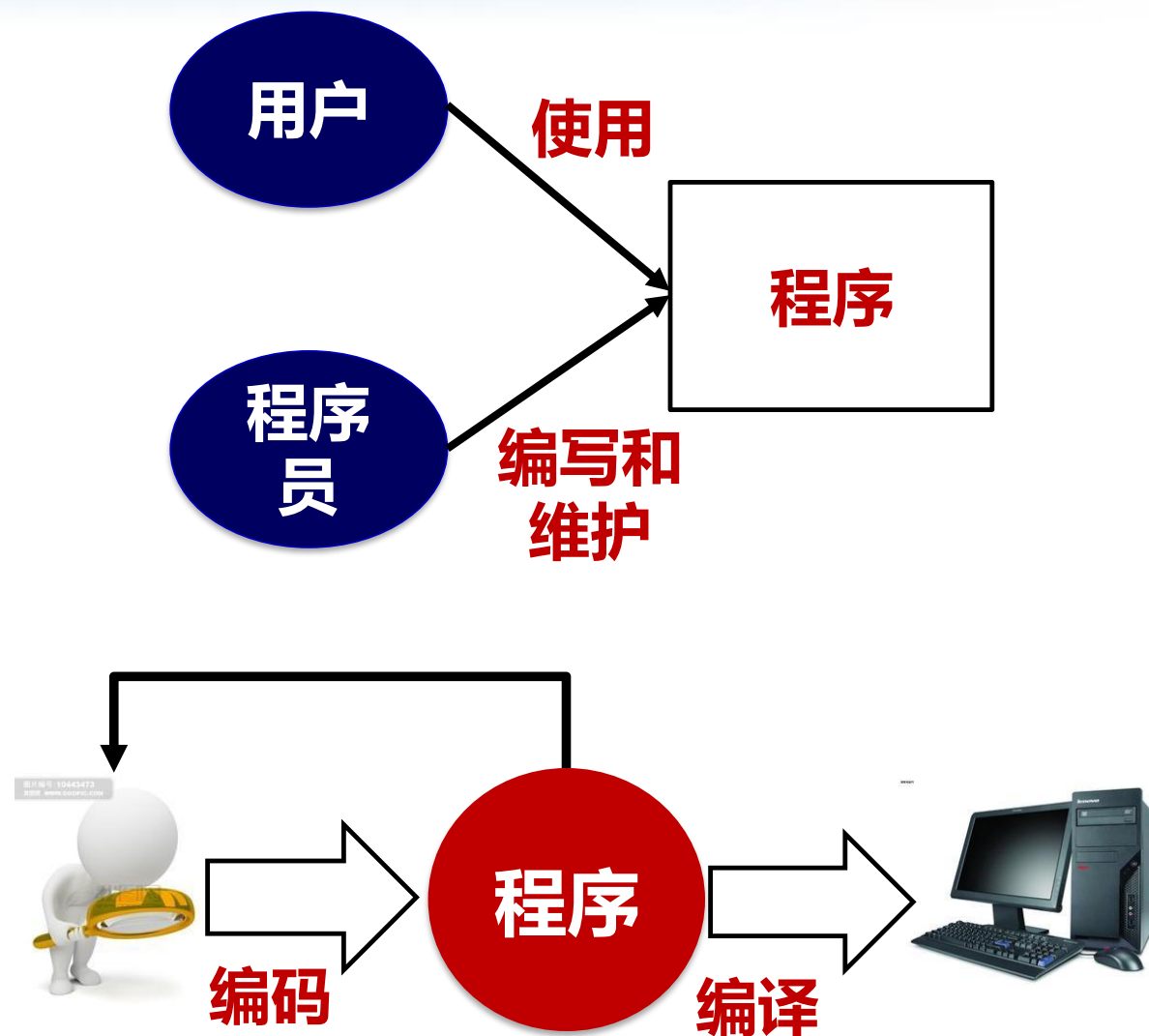
1.4 程序的利益相关者及其质量要求

□用户

- ✓程序的运行展示**功能和性能**
- ✓满足和实现用户**的需求**

□程序员

- ✓编写、阅读和维护程序
- ✓发现和修改程序中的**缺陷**

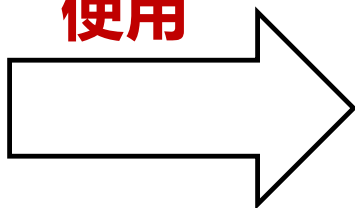


对程序的不同质量需求

用户
角度



使用



- 正确性
- 高效性
- 可靠性
- 友好性
-

程序员
角度



编写和
维护



- 可理解
- 易修改
- 可维护
- 可重用
-



用户和程序员会对程序质量分别提出什么样的要求？为什么？

程序质量的内在和外在体现

□外在质量(External Quality) – 用户的视角

- ✓ 对外可展示, 用户可直接感触到、所关心的
- ✓ 使用流畅性、响应速度、界面美观、操作简易性、运行可靠性等
- ✓ **正确性、友好性、可靠性、易用性、高效性等**

□内在质量(Internal Quality) – 程序员的视角

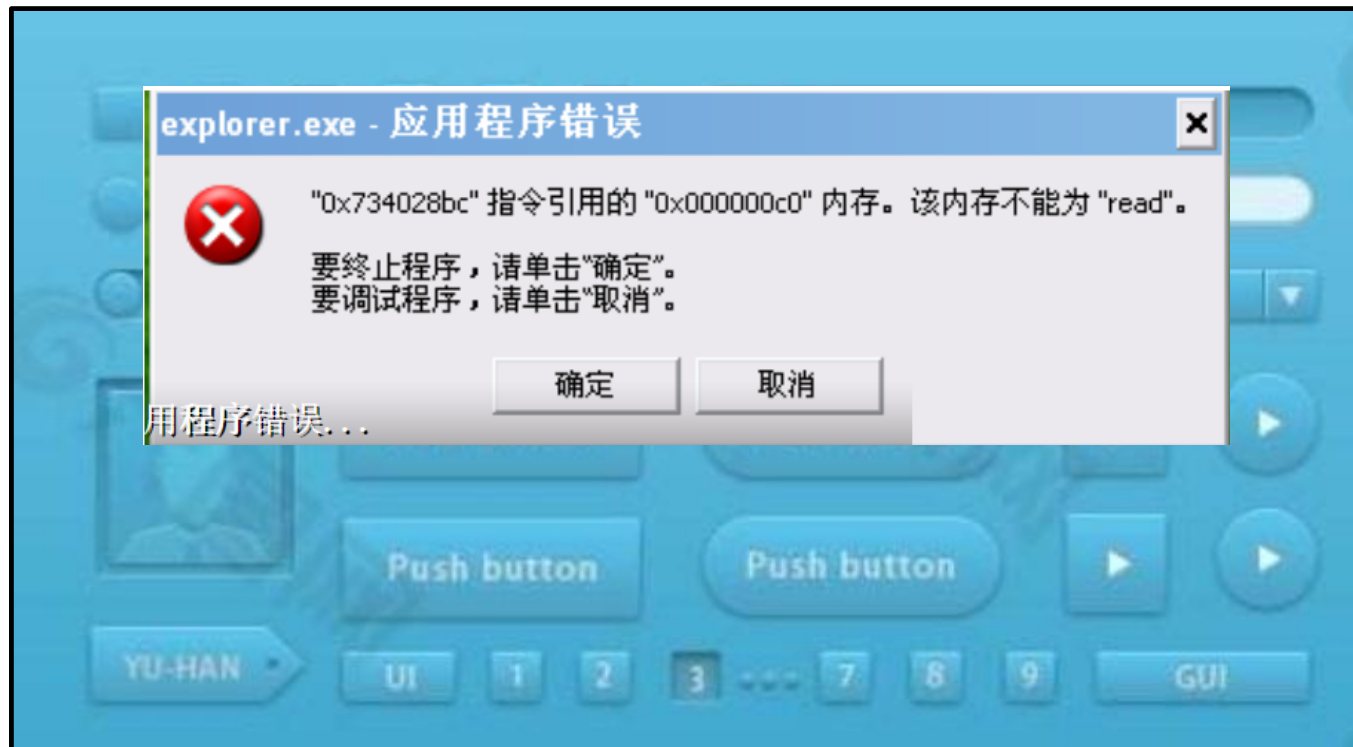
- ✓ 体现在程序的内在方面, 程序员可以感触到的、所关心的
- ✓ 易于理解、结构清晰、易于修改、可重用性好等
- ✓ **可理解性、可维护性、可扩展性、可重用性等**



你是否对你或他人编写的程序有质量方面的要求和抱怨?

示例：从用户和程序员视角看程序质量

用户角度



运行效果如何？

- ✓ 可靠吗？
- ✓ 易于操作和使用吗？
- ✓

程序员角度

```
#include "stdafx.h"
#include "stdio.h"
void test
int _tmain
(int argc,
_TCHAR* argv[])
{ test(); return
0; } char C[25]
[40]; void d(int x,
int y) { C[x][y]=
C[x][y+1]=32; }
int f(int x){return
(int)x*x*.08; }
void test(){int i,j;
char s[5]="TEST";
for(i=0;i<25;i++)
for(j=0;j<40;j++)
C[i][j]=s[(i+j)%4];
for(i=1;i<=7;i++)
{d(18-i,12);
C[20-f(i)][i+19]=
C[20-f(i)][20-i]=32;
}d(10,13);d(9,13);
d(8,14);d(7,15);
d(6,16);d(5,18);d(5,20);
d(5,22);d(5,26);
d(6,23);d(6,25);d(7,25);for(i=0;i<25;i++,printf("\n"))
for(j=0;j<40;printf("%c",C[i][j++]));
scanf("%c",&s[0]);
}
```

这段代码如何？

- ✓ 易于理解吗？
- ✓ 易于修改吗？
- ✓ 结构清晰吗？
- ✓

讨论：程序的质量

```
1 package MarDetector;
2
3
4+ import jade.core.Agent;
10
11 public class DigMineRobot extends BasicRobot {
12     private Coordinate startposi = null;
13     private MarUI ui = null;
14     private Mine mine = null;
15
16
17- public DigMineRobot() {
18     ui = MarUI.getUI();
19     mine = Mine.getMine();
20     startposi = new Coordinate(MarUI.GRID_SIZE - 1, MarUI.GRID_SIZE - 1);
21 }
22
23
24
25- public void setup() {
26     System.out.println("开始采矿!");
27     ui.runInfo.setText(ui.runInfo.getText() + "开始采矿!" + "\n");
28
29     addBehaviour(new CooperationBev(this));
30     ACLMessage msg = this.receive();
31     addBehaviour(new DigMineBev(this));
32     addBehaviour(new DumpMineBev(this));
33
34 }
35 }
36 }
37
```

这段程序写的如何?

- ✓ 正确吗?
- ✓ 易于理解吗?
- ✓ 结构清晰吗?
- ✓ 易于修改吗?

.....



思考和讨论

- 你觉得什么样的程序是高质量？
- 你编写程序时是否考虑了质量问题？
- 你所写的程序质量如何？
- 如何才能编写出高质量的程序？



内容

1. 程序及质量要求

✓程序及其内部和外部质量

2. 程序质量保证方法

✓编码规范、设计方法、代码重用、结对编程

3. 程序质量的分析方法

✓人工审查、自动化分析、代码测试

4. 编写程序需要解决的问题

✓需要工程化的方法支持



2.1 程序质量的语法和语义体现

□代码风格规范-语法

- ✓语法和结构层次
- ✓明确如何来规范程序的书写
- ✓表现为是否易于阅读和理解

□代码设计规范-语义

- ✓语义和内涵层次
- ✓明确如何来组织和封装程序语句
- ✓表现为良好的结构和易于重用

```
1 package MarDetector;  
2  
3  
4 import jade.core.Agent;  
10  
11 public class DigMineRobot extends BasicRobot {  
12     private Coordinate startposi = null;  
13     private MarUI ui = null;  
14     private Mine mine = null;  
15  
16  
17     public DigMineRobot() {  
18         ui = MarUI.getUI();  
19         mine = Mine.getMine();  
20         startposi = new Coordinate(MarUI.GRID_SIZE - 1, MarUI.GRID_SIZE - 1);  
21     }  
22  
23  
24  
25     public void setup() {  
26         System.out.println("开始采矿!");  
27         ui.runInfo.setText(ui.runInfo.getText() + "开始采矿!" + "\n");  
28  
29         addBehaviour(new CooperationBev(this));  
30         ACLMessage msg = this.receive();  
31         addBehaviour(new DigMineBev(this));  
32         addBehaviour(new DumpMineBev(this));  
33  
34  
35     }  
36 }  
37
```

程序质量保证方法

- 遵循编码风格
- 采用程序设计方法
- 开展代码重用
- 进行结对编程

2.2 遵循编码风格

□良好的编程行为

- ✓在编码过程中，程序员对代码符号进行良好的组织、合理的命名、提供必要的注释，那么将可增强代码的可读性和可理解性，进而提高代码的可维护性和可重用性，提升代码的内部质量

□何为编码风格

- ✓程序员在编码时要遵循特定的样式及要求，以规范程序员的编程行为以及所产生程序代码的样式

示例：遵循编码风格

```
#include "stdafx.h"
#include "stdio.h"
void test
    ();
int _tmain
    (int argc,
     _TCHAR* argv[])
{ test(); return
  0; } char C[25]
[40]; void d(int x,
int y) {C[x][y]=
C[x][y+1]=32;}
int f(int x){return
(int)x*x*.08;}
void test(){int i,j;
char s[5]="TEST";
for(i=0;i<25;i++)
for(j=0;j<40;j++)
C[i][j]=s[(i+j)%4];
for(i=1;i<=7;i++)
{d(18-i,12);
C[20-f(i)][i+19]=
C[20-f(i)][20-i]=32;
}d(10,13);d(9,13);
d(8,14);d(7,15);
d(6,16);d(5,18);d(5,20);          d(5,22);d(5,26);
d(6,23);d(6,25);d(7,25);for(i=0;i<25;i++,printf("\n"))
for(j=0;j<40;printf("%c",C[i][j++]));
scanf("%c", &s[0]);
}
```

没有遵循编码风格

```
1  /*
2   * Copyright (c) 2010-2011, The MiCode Open Source Community (www.micode.net)
3   *
4   * Licensed under the Apache License, Version 2.0 (the "License");
5   * you may not use this file except in compliance with the License.
6   * You may obtain a copy of the License at
7   *
8   *      http://www.apache.org/licenses/LICENSE-2.0
9   *
10  * Unless required by applicable law or agreed to in writing, software
11  * distributed under the License is distributed on an "AS IS" BASIS,
12  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13  * See the License for the specific language governing permissions and
14  * limitations under the License.
15  */
16
17  package net.micode.notes.data;
18
19  import android.content.Context;
20
21  public class Contact {
22      private static HashMap<String, String> sContactCache;
23      private static final String TAG = "Contact";
24
25      private static final String CALLER_ID_SELECTION = "PHONE_NUMBERS_EQUAL(" + Phone.NUMBER
26      + ",?) AND " + Data.MIMETYPE + "='" + Phone.CONTENT_ITEM_TYPE + "'"
27      + " AND " + Data.RAW_CONTACT_ID + " IN "
28      + "(SELECT raw_contact_id "
29      + " FROM phone_lookup"
30      + " WHERE min_match = '1')";
31
32      public static String getContact(Context context, String phoneNumber) {
33          if(sContactCache == null) {
34              sContactCache = new HashMap<String, String>();
35          }
36
37          if(sContactCache.containsKey(phoneNumber)) {
38              return sContactCache.get(phoneNumber);
39          }
40      }
41  }
```

遵循编码风格

编写代码的基本原则

□易读，一看就懂

- ✓理解代码的内涵和意图
- ✓望文生义的符号、缩进和括号、代码注释，遵循编码风格

□简明，减低复杂度

- ✓避繁就简、不用goto语句、减少嵌套层数、简单算法

□易改，便于维护

- ✓便于修改程序代码、增加新的代码
- ✓封装、参数化、模块化、隐藏、常元

□无二义，不产生歧义

- ✓不要让人产生误解

编码风格-代码布局

□ 缩进，用好Tab键

□ 用括号来表示优先级

□ 断行处的{ }

```
if (condition) {  
    DoSomething();  
} else {  
    DoSomething();  
}
```

□ 一行至多只一条语句

✓ 不要在一行定义多个变量

```
146 public void onDismiss(DialogInterface dialog) {  
147     stopAlarmSound();  
148     finish();  
149 }  
150  
151 private void stopAlarmSound() {  
152     if (mPlayer != null) {  
153         mPlayer.stop();  
154         mPlayer.release();  
155         mPlayer = null;  
156     }  
157 }  
158 }
```

编码风格-代码组织

- 按一定次序来说明数据
- 按字母顺序说明对象
- 尽可能避免使用嵌套结构
- 采用统一的缩进规则
- 单入口单出口

```
91     private void playAlarmSound() {
92         Uri url = RingtoneManager.getActualDefaultRingtoneUri(this,
93             RingtoneManager.TYPE_ALARM);
94
95         int silentModeStreams = Settings.System.getInt(getContentResolver(),
96             Settings.System.MODE_RINGER_STREAMS_AFFECTED, 0);
97
98         if ((silentModeStreams & (1 << AudioManager.STREAM_ALARM)) != 0) {
99             mPlayer.setAudioStreamType(silentModeStreams);
100         } else {
101             mPlayer.setAudioStreamType(AudioManager.STREAM_ALARM);
102         }
103         try {
104             mPlayer.setDataSource(this, url);
105             mPlayer.prepare();
106             mPlayer.setLooping(true);
107             mPlayer.start();
108         } catch (IllegalArgumentException e) {
109             // TODO Auto-generated catch block

```

编码风格-命名规范

□采用有意义、一目了然的命名方式

- ✓变/常/函数/方法/类/包
- ✓一看就懂，望文生义

□无意义的命名

- ✓i, j, x, y

□有意义的命名

- ✓szPath
- ✓vPrintName()
- ✓bReturn

```
38 public class NotesProvider extends ContentProvider {  
39     private static final UriMatcher mMatcher;  
40  
41     private NotesDatabaseHelper mHelper;  
42  
43     private static final String TAG = "NotesProvider";  
44  
45     private static final int URI_NOTE = 1;  
46     private static final int URI_NOTE_ITEM = 2;  
47     private static final int URI_DATA = 3;  
48     private static final int URI_DATA_ITEM = 4;  
49  
50     private static final int URI_SEARCH = 5;  
51     private static final int URI_SEARCH_SUGGEST = 6;  
52
```

编码风格-命名

□命名通则

- ✓使用**英文单词或缩写**，不要使用拼音
- ✓**望文知义**原则，含义清晰、明确
- ✓命名**不要过长**
- ✓尽量使用**全称**，少用缩写

■不规范的命名

- ✓DaYinWenJian
与 PrintFile

```
public class LoginManager{  
    private UserLibrary userLib;   
      
    public LoginManager( );//构造函数  
    public int login(account, password); //用户登录  
    public boolean isUserValid(String account, String password); //判断用户是否合法  
}
```

编码风格-命名

□命名和大小写

- ✓类/类型/变量：**用名词和名词短语**，所有单词第一个字母大写
- ✓函数/方法：**动词或者动词短语**，第一个单词小写，随后单词第一个字母大写
- ✓示例：Member, ProductInfo, getName(), setName(), renderPage()

□不规范的命名

- ✓变量：print;
- ✓方法：Print()

```
public class LoginManager {  
    private UserLibrary userLib;   
      
    public void LoginManager( ); //构造函数  
    public int login(account, password); //用户登录  
    public boolean isUserValid(String account, String password); //判断用户是否合法  
}
```

编码风格-代码注释

□ 帮助理解程序

- ✓ 注释要说明程序：(1) **做什么**；(2) **为什么这么做**；(3) **注意事项**
- ✓ 无需解释程序如何做

□ 注解位置

- ✓ **类头、函数/函数头、关键语句块头、关键语句尾**

□ 有效、必要、简洁的注释

- ✓ 太少和过多均不可取
- ✓ 注释要可理解、准确、无二义

□ 随代码的修改而修改

```
1  /*
2   * Copyright (c) 2010-2011, The MiCode Open Source Community (www.micode.net)
3   *
4   * Licensed under the Apache License, Version 2.0 (the "License");
5   * you may not use this file except in compliance with the License.
6   * You may obtain a copy of the License at
7   *
8   *     http://www.apache.org/licenses/LICENSE-2.0
9   *
10  * Unless required by applicable law or agreed to in writing, software
11  * distributed under the license is distributed on an "AS IS" BASIS,
12  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13  * See the License for the specific language governing permissions and
14  * limitations under the License.
15  */
16
17  package net.micode.notes.tool;
18
19  import android.content.Context;
20
21
22  public class BackupUtils {
23      private static final String TAG = "BackupUtils";
24      // Singleton stuff
25      private static BackupUtils sInstance;
26
27      public static synchronized BackupUtils getInstance(Context context) {
28          if (sInstance == null) {
29              sInstance = new BackupUtils(context);
30          }
31          return sInstance;
32      }
33
34      /**
35       * Following states are signs to represents backup or restore
36       * status
37       */
38      // Currently, the sdcard is not mounted
39      public static final int STATE_SD_CARD_UNMOUNTED = 0;
```

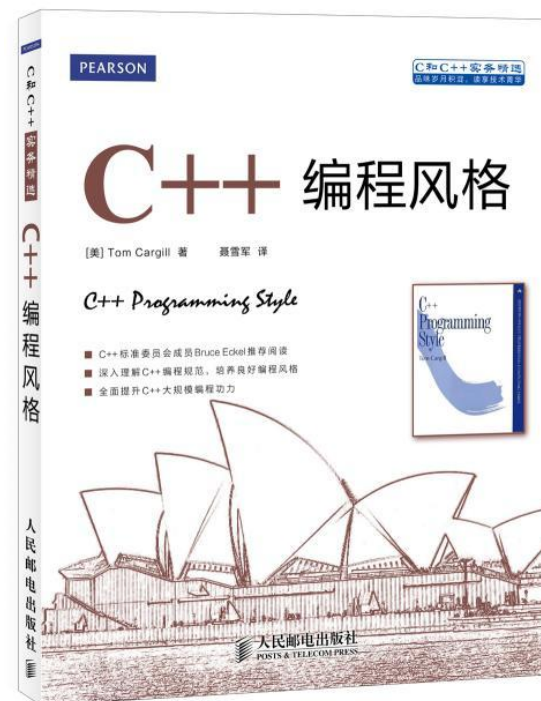
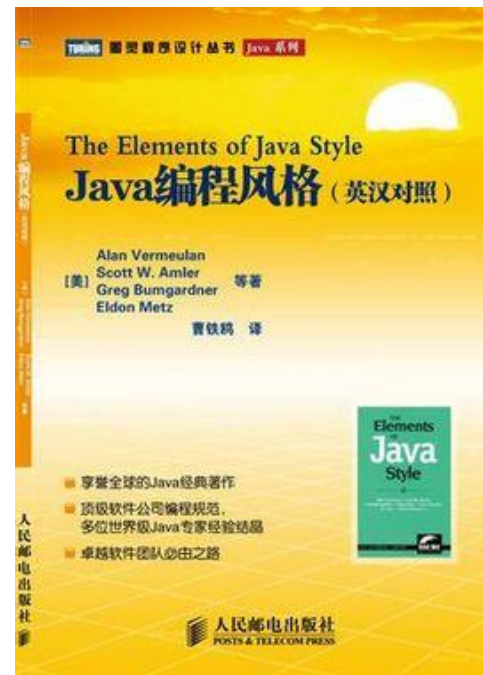
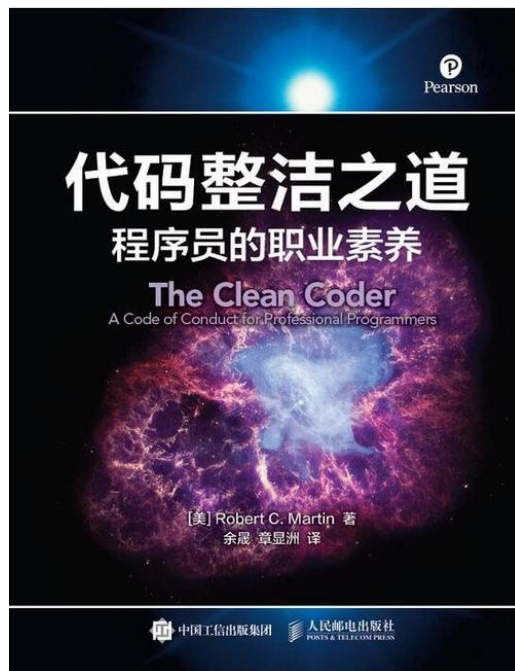
编码风格示例

```
1  /*
2   * Copyright (c) 2010-2011, The MiCode Open Source Community (www.micode.net)
3   *
4   * Licensed under the Apache License, Version 2.0 (the "License");
5   * you may not use this file except in compliance with the License.
6   * You may obtain a copy of the License at
7   *
8   *     http://www.apache.org/licenses/LICENSE-2.0
9   *
10  * Unless required by applicable law or agreed to in writing, software
11  * distributed under the License is distributed on an "AS IS" BASIS,
12  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13  * See the License for the specific language governing permissions and
14  * limitations under the License.
15  */
16
17 package net.micode.notes.data;
18
19 import android.content.Context;
20
21 public class Contact {
22     private static HashMap<String, String> sContactCache;
23     private static final String TAG = "Contact";
24
25     private static final String CALLER_ID_SELECTION = "PHONE_NUMBERS_EQUAL(" + Phone.NUMBER
26     + ",?) AND " + Data.MIMETYPE + "='" + Phone.CONTENT_ITEM_TYPE + "'"
27     + " AND " + Data.RAW_CONTACT_ID + " IN "
28     + "(SELECT raw_contact_id "
29     + " FROM phone_lookup"
30     + " WHERE min_match = '+')";
31
32     public static String getContact(Context context, String phoneNumber) {
33         if(sContactCache == null) {
34             sContactCache = new HashMap<String, String>();
35         }
36
37         if(sContactCache.containsKey(phoneNumber)) {
38             return sContactCache.get(phoneNumber);
39         }
40     }
41 }
```

✓ 注释
✓ 命名
✓ 布局
✓ 结构

编码风格的相关书籍

- C++ 编程风格
- Java 编程风格
- 代码整洁之道
- 推崇 Clean Code



不同程序设计语言有不同的编码风格要求

讨论：编码风格

- ✓ 这二段代码风格如何？哪些做的好，还存在哪些问题？
- ✓ 你编写代码时注意风格了吗？

```
1 package com.spring.menu.animation;
2
3 import android.view.View;
4 import android.view.animation.AlphaAnimation;
5 import android.view.animation.ScaleAnimation;
6
7 public class EnlargeAnimationIn extends ZoomAnimation {
8     public EnlargeAnimationIn(int i) {
9         super(ZoomAnimation.Direction.HIDE, i, new View[0]);
10    }
11
12    @Override
13    protected void addShrinkAnimation(View[] views) {
14        // TODO Auto-generated method stub
15        addAnimation(new ScaleAnimation(0F, 1F, 0F, 1F, 1, 0.5F, 1, 0.5F));
16        addAnimation(new AlphaAnimation(0F, 1F));
17    }
```

```
private void showLinearMenus()
{
    int[] size = DeviceUtility.getScreenSize(this);

    if (!areMenusShowing) {
        SpringAnimation.startAnimations(
            this.menusWrapper, ZoomAnimation.Direction.SHOW, size);
        //this.imageViewPlus.startAnimation(this.animRotateClockwise);
    } else {
        SpringAnimation.startAnimations(
            this.menusWrapper, ZoomAnimation.Direction.HIDE, size);
        //this.imageViewPlus.startAnimation(this.animRotateAntiClockwise);
    }
}
```



2.3 采用程序设计方法

- 语句设计
- 模块化设计
- 高内聚度、低耦合度原则

代码设计规范-语句设计

- 单入口单出口

- 少用goto语句

- 加强对异常处理

- ✓分析和验证输入参数正确性，如名字不为空，年龄大于0少于150

- ✓执行结果处理，设置必要断言来分析

- 如 `p = AllocateNewSpace( );`

- `Assert(p != NULL);`

- `if (p != NULL) {`

- `.....`

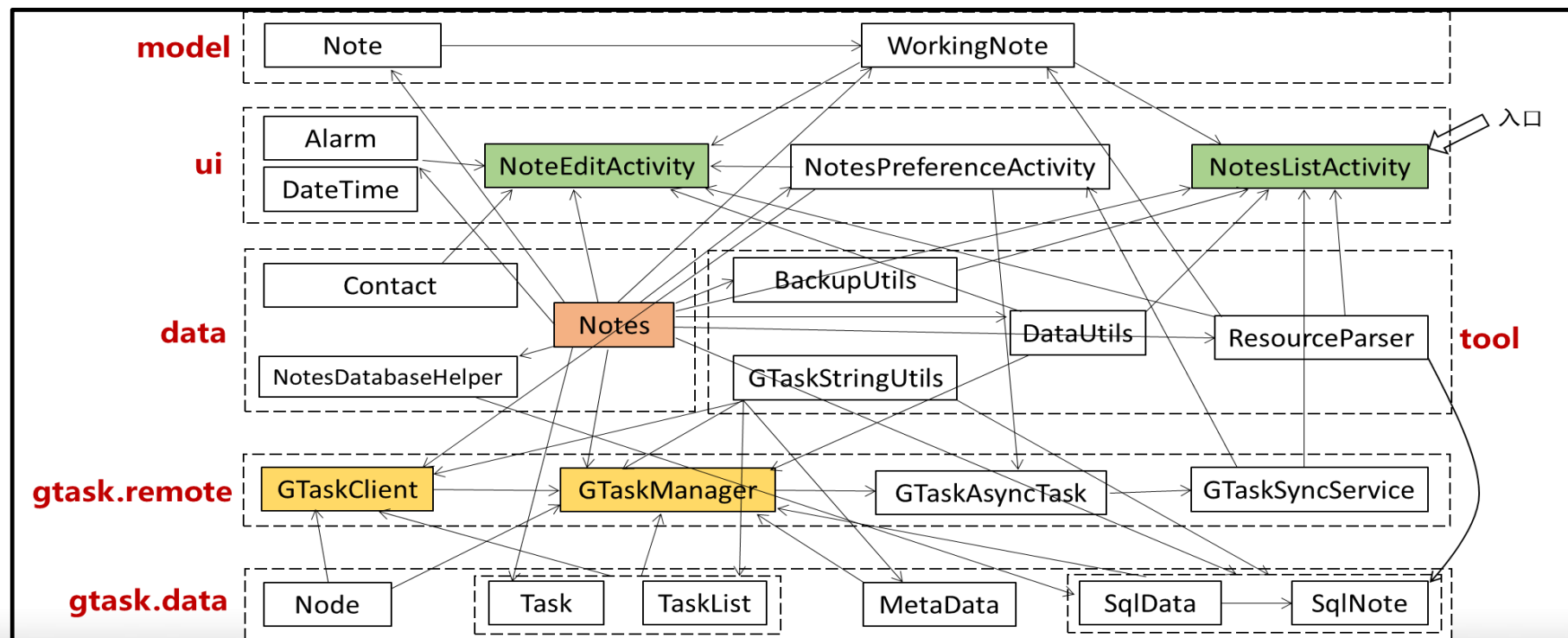
- `}`

- ✓处理异常语句 `Try { ... } catch (Exception e) { ... }`

将更多的时间、精力和代码放在处理异常上

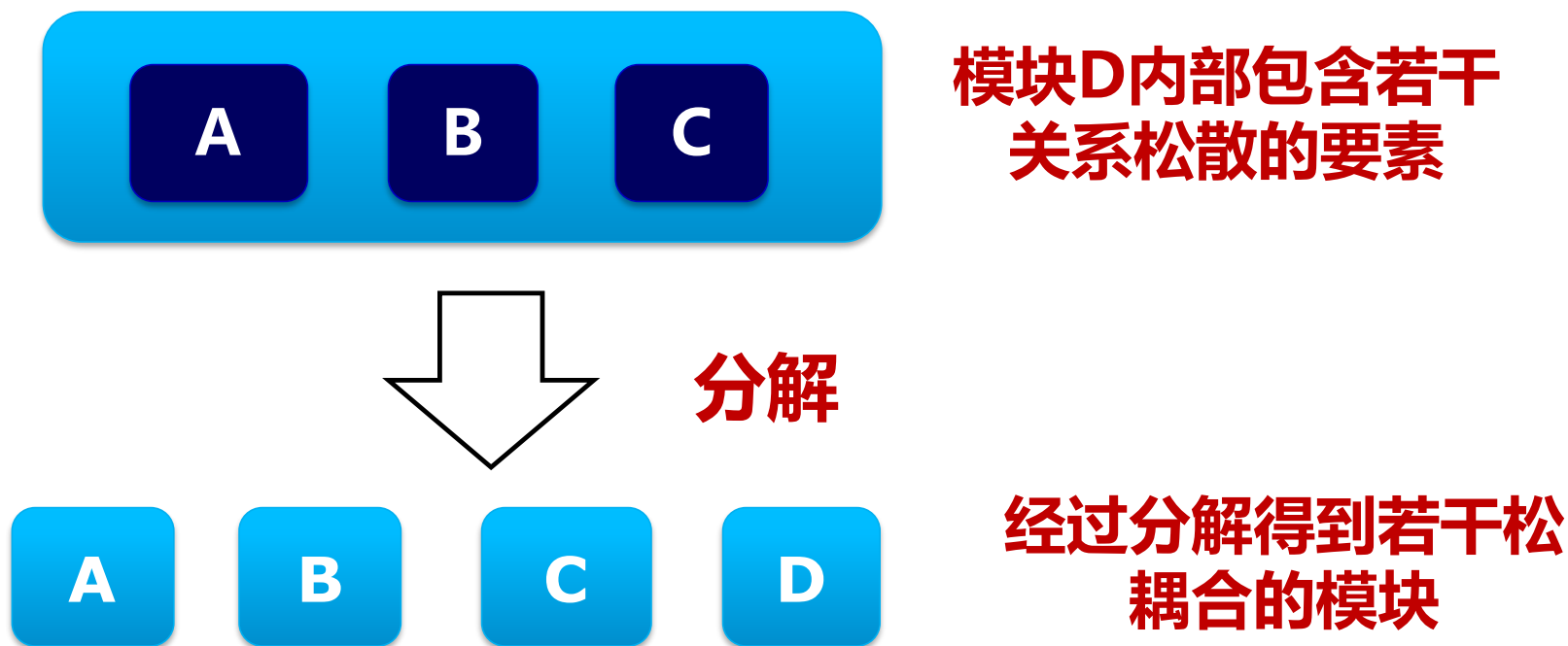
代码设计规范-模块化设计

- 模块是逻辑上相对**独立**、具有**良定义接口**的编程单位
- 模块可表现为**函数**、**过程**、**方法**、**类**、**程序包**等
- 将程序代码封装为一个个独立的模块



代码设计规范-高内聚度

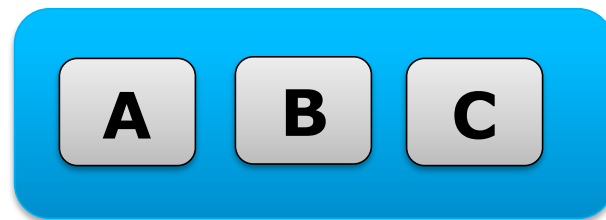
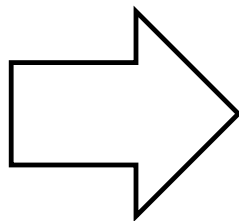
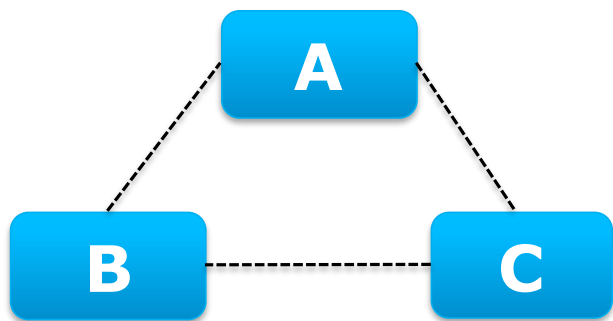
- 模块内各要素**紧密相关**，仅实现**单一功能**
- 如果模块内多个要素关系不密切，需分解产生多个模块



代码设计规范-低耦合度

- 模块间的关系应设计的非常松散
- 如果多个模块间的关系非常密切，可将这些模块合并为一个模块

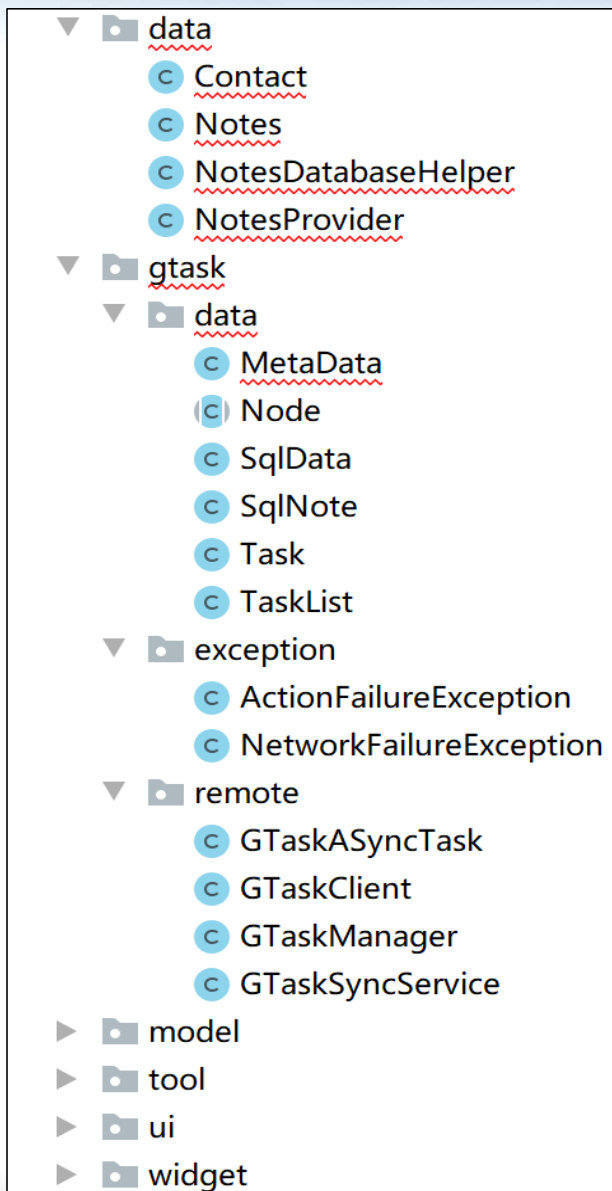
若干关系紧密的模块



经过合并得到内聚度高的模块D

示例：采用程序设计方法

- 语句设计
- 模块化
- 高内聚度
- 低耦合度



```
package net.micode.notes.data;

import ...

public class NotesProvider extends ContentProvider {
    private static final UriMatcher mMatcher;

    private NotesDatabaseHelper mHelper;

    private static final String TAG = "NotesProvider";

    private static final int URI_NOTE = 1;
    private static final int URI_NOTE_ITEM = 2;
    private static final int URI_DATA = 3;
    private static final int URI_DATA_ITEM = 4;

    private static final int URI_SEARCH = 5;
    private static final int URI_SEARCH_SUGGEST = 6;

    static {
        mMatcher = new UriMatcher(UriMatcher.NO_MATCH);
        mMatcher.addURI(Notes.AUTHORITY, "note", URI_NOTE);
        mMatcher.addURI(Notes.AUTHORITY, "note/#", URI_NOTE_ITEM);
    }
}
```

2.4 代码重用

□何为代码重用

- ✓在编写代码过程中，**充分利用已有和现成的代码**，并将其集成到程序之中，从而来实现程序功能

□代码重用有何好处

- ✓由于被重用的代码经过多次反复的使用，代码质量得到充分检验，因而代码重用不仅可**提高编程效率**，而且还可有效**提升程序质量**



为什么代码重用可以提高代码的质量？

示例：代码片段重用

- 寻找他人实现某些子功能的**代码片段**
- 可在Stack Overflow上寻找到**有价值的代码片段**，并加以重用

```
private Database(LSE item) {  
    ric = item.get_ric();  
    volume = item.get_volume();  
}  
  
public static final Database getInstance(LSE item) {  
    if (INSTANCE == null) {  
        INSTANCE = new Database(LSE item);  
    }  
  
    return INSTANCE;  
}  
  
public void writeToDb() throws SQLException{  
    //setString  
}  
}
```

If your application will be using Threads (Concurrency), I suggest you also to prepare your singleton for those situations , [see this question](#)

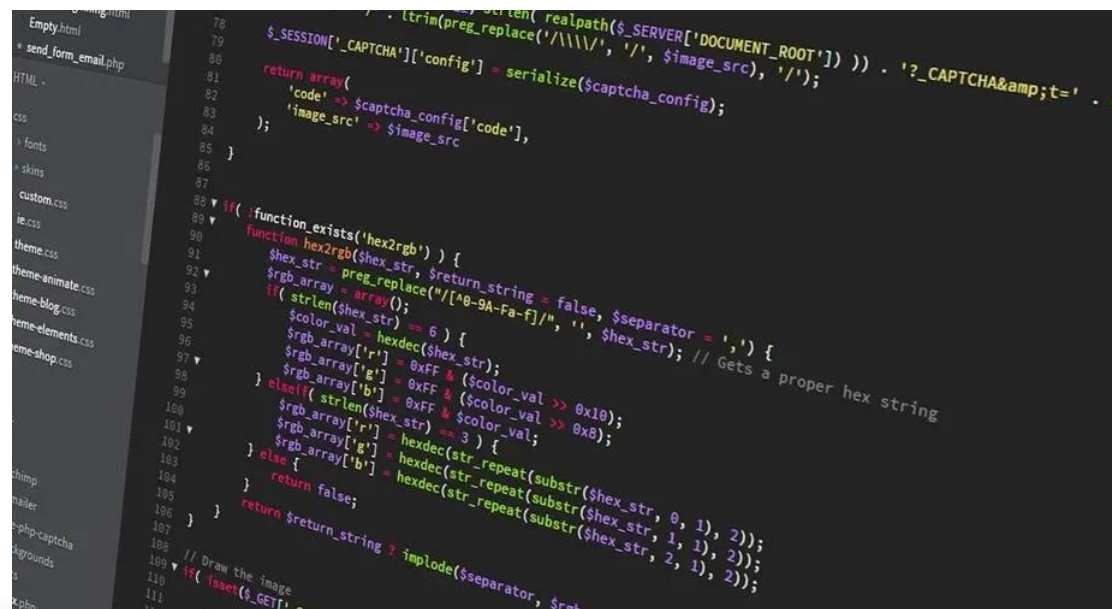
开源代码重用

❑到Github中找到粗粒度的代码开源

✓几万、甚至几十万的程序代码

□重用开源代码来实现粗粒度的功能

✓完成诸如数据库管理、图像识别、语音分析等功能



思考和讨论

- 你在编程时是否有软件重用？
- 你开展了那些方面和粒度的软件重用？



2.5 结对编程 (Pair-Programming)

□ **两位程序员（或智能体）** 坐在同一工作台前一起**开发软件**

- ✓ 一人扮演“领航员”角色，负责具体编写工作，如写程序
- ✓ 一人扮演“观察者”角色，负责观察行为及结果，如看程序，发现问题
- ✓ 二者相互讨论，共同完成编程任务



大模型时代基于智能体的结对编程

越来越多程序员使用CopilotX等智能化工具进行软件开发，**AI Pair 编程**正在成为新的编程范式，帮助程序员编代码、单元测试、调试、写文档等



软件工程师



人机协同开发



软件开发智能体

基于智能化的编程助手（如CopilotX），人机协同编程会成为趋势

个体开发的局限性

□个体知识和技能的局限性

- ✓没有人无所不能
- ✓有专长，但很难做到面面俱到
- ✓总会有不懂、不会的

□个体开发行为的局限性

- ✓人总是会犯错误的，错误很难避免
- ✓老虎也有打盹的时候，注意力不集中就会犯错误
- ✓很难看到或者看清自身的错误

软件开发是集体性/群体性行为

□团队开发

- ✓多人参与、具有共同目的、明确任务分工

□合作开发

- ✓相互支持、互相配合、共同解决问题

□群体开发

- ✓利用大众的力量、借助大众的智慧和成果

□抛弃个人主义/英雄主义

- ✓单枪匹马难成大事、单干/蛮干容易出事

现实世界的结对示例



✓ 驾驶员
✓ 领航员



✓ 驾驶员
✓ 作战员

如何实现结对编程

□职责明确

- ✓一人写设计文档、编写程序和单元测试等等
- ✓一人审阅文档、复审程序代码、考虑单元测试的覆盖率、是否需要重构、解决具体的技术问题等等

□互换角色

- ✓不要连续超过工作1小时，提高效率

□主动参与

- ✓开展讨论、解决问题、做出贡献

编程行为及其特点

□快速完成

- ✓尽快得到可运行软件系统、尽早交付给用户、快速应对需求变化

□多种技能

- ✓工具和环境
- ✓程序设计语言
- ✓业务领域知识
- ✓编码规范和设计规范

□质量要求

- ✓正确性、可读性、可维护性等



编程包袱：效率和质量问题

结对写程序

□任务

- ✓编写程序代码

□方式

- ✓一人写一人复审
- ✓规范性、正确性、可读性等

□讨论

- ✓改进与提高
- ✓如增加注释、更改名称等



结对编程中的代码复审

- 结对编程是一个不断“**复审**”代码的过程
- 每一行代码都被**二双眼睛**看过，被**二个脑子**思考过
- 代码随着改动**不断地被复审**
- 每个人的一举一动（编码行为）不断地被另一个人审查，确保过程和活动置于**监督**之下，迫使认真工作，防止随意行为
- 促进“**频繁**”**交流**，提高个人能力和素养

结对写文档

□任务

- ✓撰写文档

□方式

- ✓一人写一人复审
- ✓规范、正确、合理性等

□讨论

- ✓改进和完善
- ✓如文档格式、语言表达、图表、错别字



结对做测试

□任务

- ✓软件测试

□方式

- ✓运行测试用例，收集测试结果
- ✓一人写一人复审和帮助
- ✓完整性、代表性、适当性等

□讨论

- ✓完善和提升



结对编程带来的好处

□提高程序质量

- ✓ 提供更好的设计质量和代码质量
- ✓ 合作解决问题能力强, $1+1 > 2$

□提升开发效率

- ✓ 开发人员更加信心
- ✓ 有效地避免了闭门造车
- ✓ 更易于发现问题和纠正问题

□促进学习交流

- ✓ 有效的学习, 做中学效果更好
- ✓ 相互学习和分享经验
- ✓ 更好应对人员流动, 一个走了另一个人可以替换上

结对编程可以获得更高的投入/产出比

思考和讨论

- 如何在阅读、分析和维护开源软件实践中应用结对编程的方法？
- 有何实践经验可供分享？



内容

1. 程序及质量要求

- ✓ 程序及其内部和外部质量

2. 程序质量保证方法

- ✓ 编码规范、设计方法、代码重用、结对编程

3. 程序质量的分析方法

- ✓ 人工审查、自动化分析、代码测试

4. 编写程序需要解决的问题

- ✓ 需要工程化的方法支持



3.1 程序代码中潜在的质量问题

□质量问题

- ✓ **编写不合理** – 没有遵循编码规范
- ✓ **设计不合理** – 没有遵循设计规范
- ✓ **代码有错误** – 代码编写的不正确

□原因

- ✓ 受软件开发人员**经验和水平**的限制
- ✓ 人可能会犯错误, **人为引入错误**



你编写的代码是否存在问题？存在哪些方面的问题？

代码质量分析的常用方法

- 人工审查(Code Review)方法
- 自动化分析方法(Automated Analysis)
- 程序测试 (Test) 方法

3.2 人工审查代码

□方法描述

- ✓ 阅读和理解代码
- ✓ 发现缺陷和问题
- ✓ 提出改进的建议



□方法特点

- ✓ 人工审查效率低
- ✓ 难以发现一些深层次问题
- ✓ 难以全面地进行系统分析

```
1 package MarDetector;
2
3 import java.awt.Image;
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21 /*构建类: 基础机器人; 属性: 当前位置, 方法: 在网格上由当前位置单步移动到另外一个位置*/
22 public class BasicRobot extends Agent {
23     /* 基础机器人的位置 */
24     public Coordinate posi = null;
25     private BasicRobot basicrobot = null;
26
27     public BasicRobot(){
28         basicrobot = new BasicRobot();
29     }
30     /*设置机器人位置*/
31     public void setRobotPosi(Coordinate p){
32         posi = p;
33     }
34
35     /*获取机器人位置*/
36     public Coordinate getRobotPosi(BasicRobot thisrobot){
37         return(thisrobot.posi);
38     }
39     /* 基础机器人由当前位置单步移动到另外一个位置方法实现 */
40     public void moveStep(Coordinate start, Coordinate next) {
41         ShortestPath path = new ShortestPath();
42         path.setStart(start.x, start.y);
43         path.setTarget(next.x, next.y);
44     }
45
46
47     protected void setup() {
48         System.out.println("BasicRobot name is " + getLocalName());
49         addBehaviour(new RandomWalkBev(this));
50     }
51
52
53 }
54
```

人工审查些什么？

□ 代码是否符合编程规范

□ 代码中是否存在缺陷

- ✓ 逻辑错误， “+” 写成 “-”
- ✓ 算法错误， 不够优化、边界条件没有处理好
- ✓ 潜在错误， 当前修改导致以前修复的错误重现

□ 从质量的角度哪些代码需要改进

- ✓ 包括内部质量和外部质量

□ 读别人编写的高质量代码能让你受益匪浅

□ 读低质量的代码能让你非常痛苦

人工审查-谁负责审查

- **自我复审**，效果不一定好
- **同伴复审**，常用方法 → **结对编程**
- **团队复审**，团队成员参加

3.3 自动化工具审查-代码静态分析

□由计算机软件来自动完成代码审查

- ✓ **无需运行被测代码**，仅通过分析或检查程序的语法、结构、过程、接口等来检查程序
- ✓ 30% - 70% 的代码缺陷可通过静态分析发现

□分析什么

- ✓ **找出代码隐藏的 errors 和缺陷**，如参数不匹配，有歧义的嵌套语句，错误的递归，非法计算，可能出现的空指针引用等等
- ✓ **程序遵循编码规范的程度**

□特点

- ✓ 快速定位、有效发现隐藏的 errors 和缺陷

自动化代码分析工具

❑ SonarQube

❑ CheckStyle

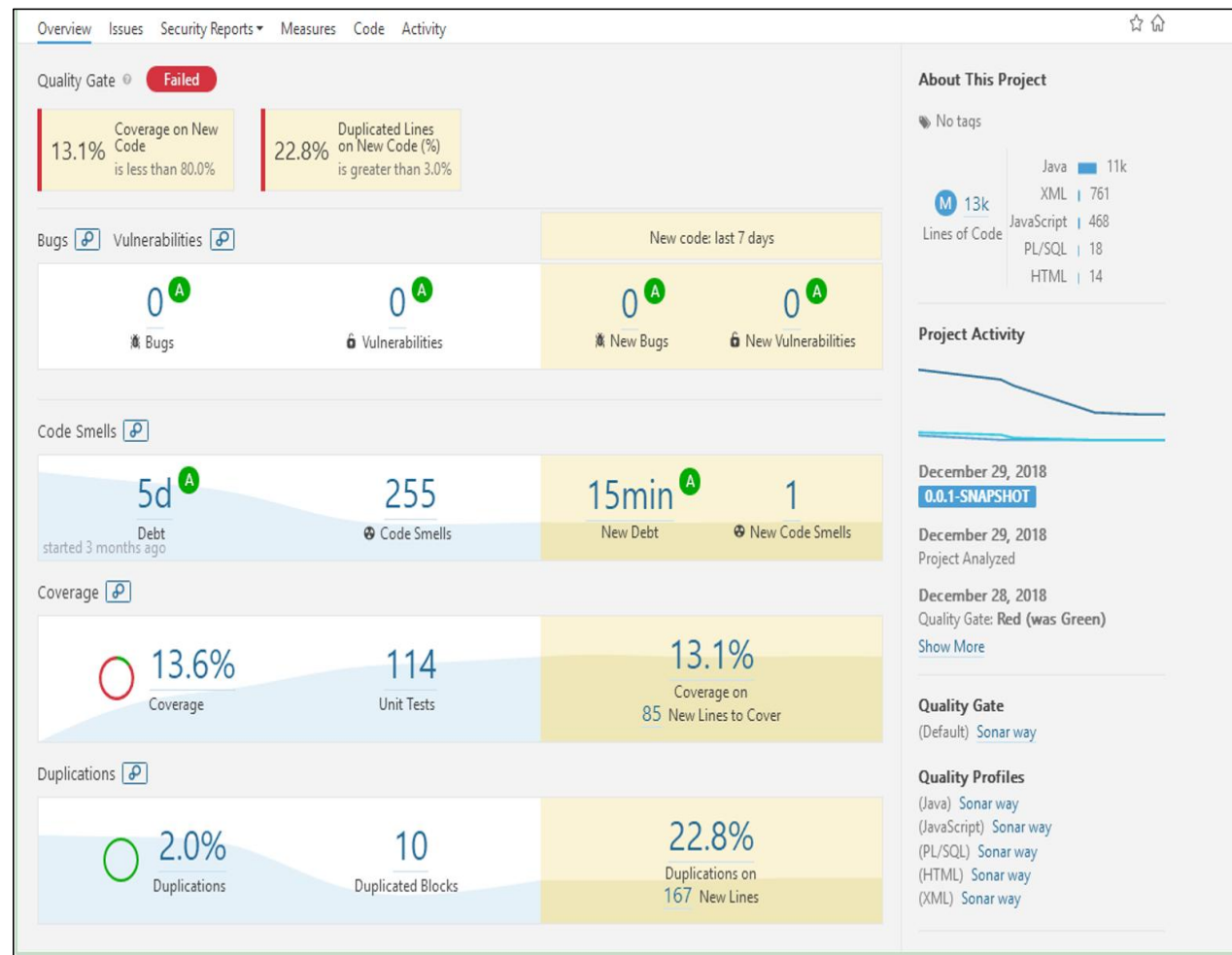
❑ FindBugs

❑ PMD

❑ Jtest

❑ CodeArts Check(华为)

❑

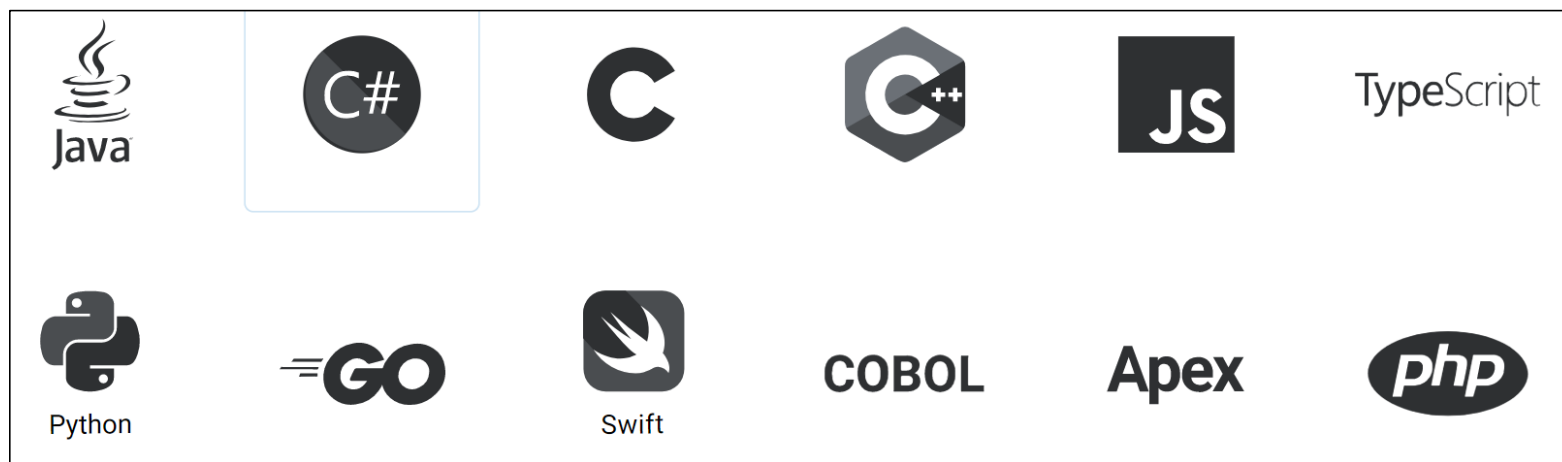


SonarQube概述

□基于Web、用于管理程序代码质量的**代码静态分析工具**

- ✓ www.sonarqube.org
- ✓ 可以插件的形式集成到众多的软件开发环境（如Eclipse）

□支持二十多种程序设计语言代码的质量分析



sonarqube 

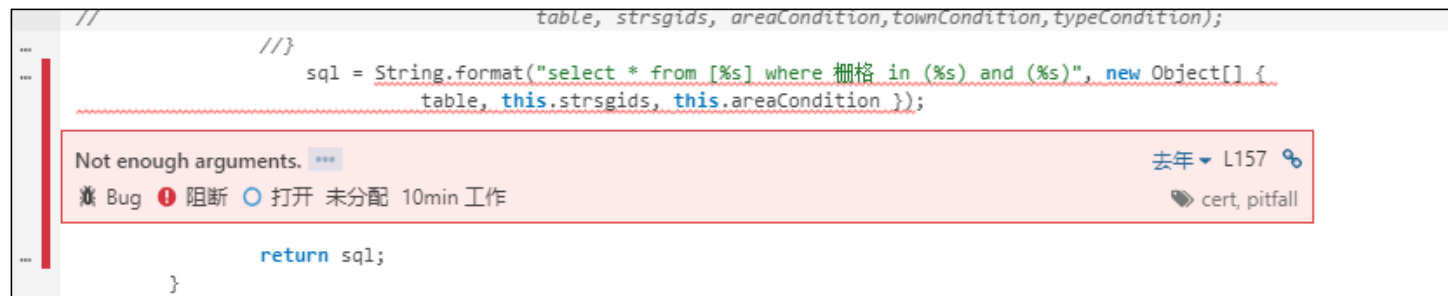
SonarQube可分析的质量问题

- 是否违反**编码规则**
- 是否存在静态**常规缺陷**
- 模块、方法、类的**复杂度**是否过高
- 是否存在**重复的代码**
- 代码的**注释**是否恰当和充分
- 统计和分析代码的**单元测试覆盖率**
- 判断软件体系结构设计是否**合理**

SonarQube分析发现的问题类别

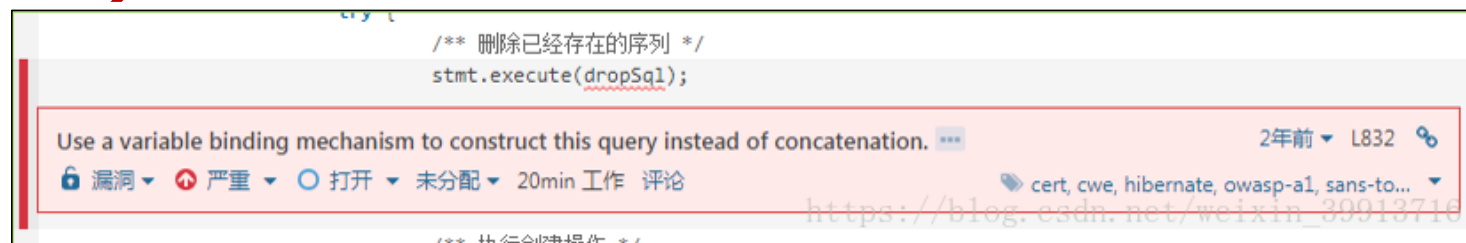
❑ Bug (错误)

- ✓ 中等影响
- ✓ 如参数不够



❑ Vulnerability (脆弱点)

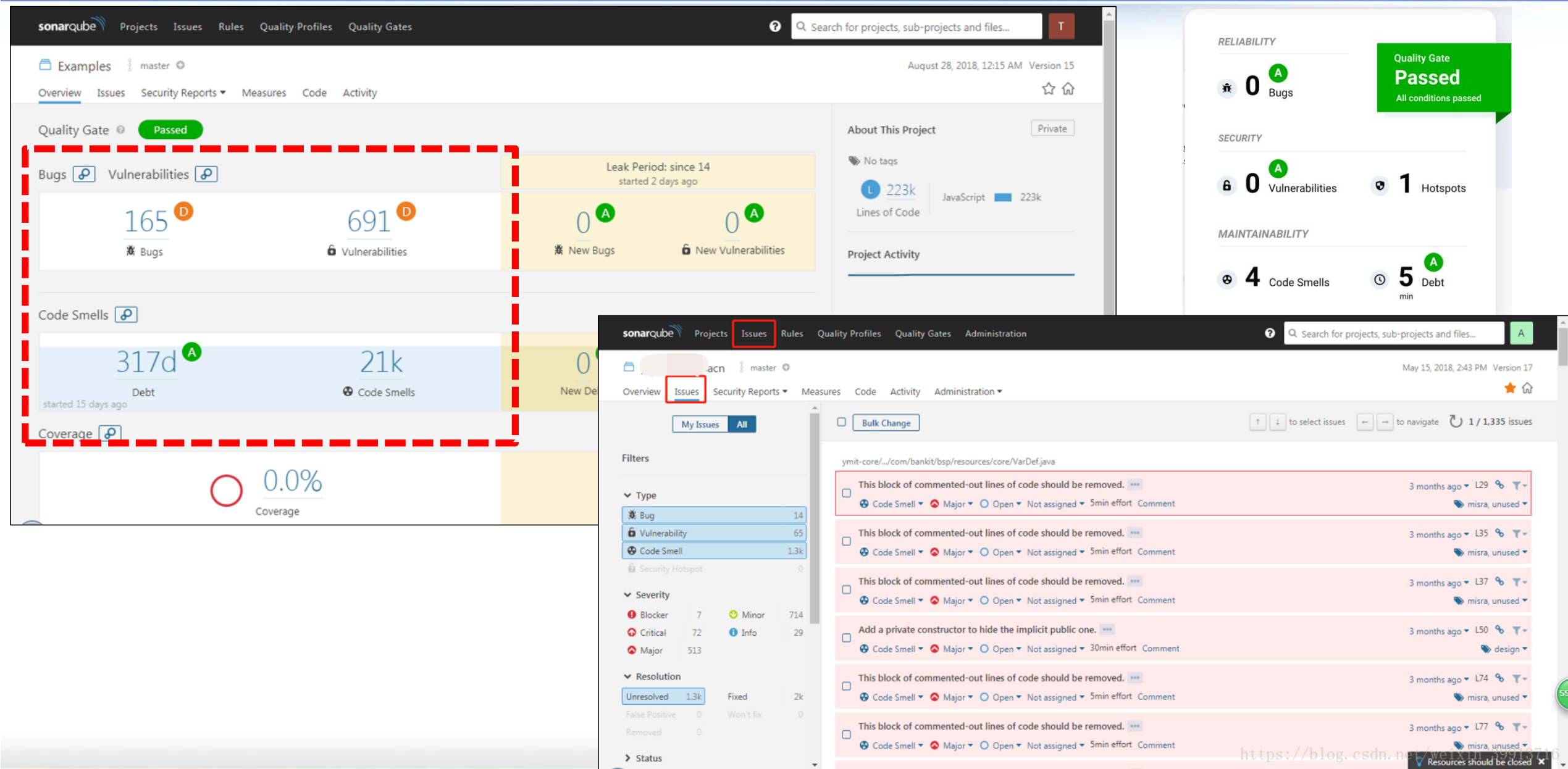
- ✓ 影响大
- ✓ 如漏洞



❑ Code Smell (代码异味)

- ✓ 影响小
- ✓ 如风格

SonarQube分析报告

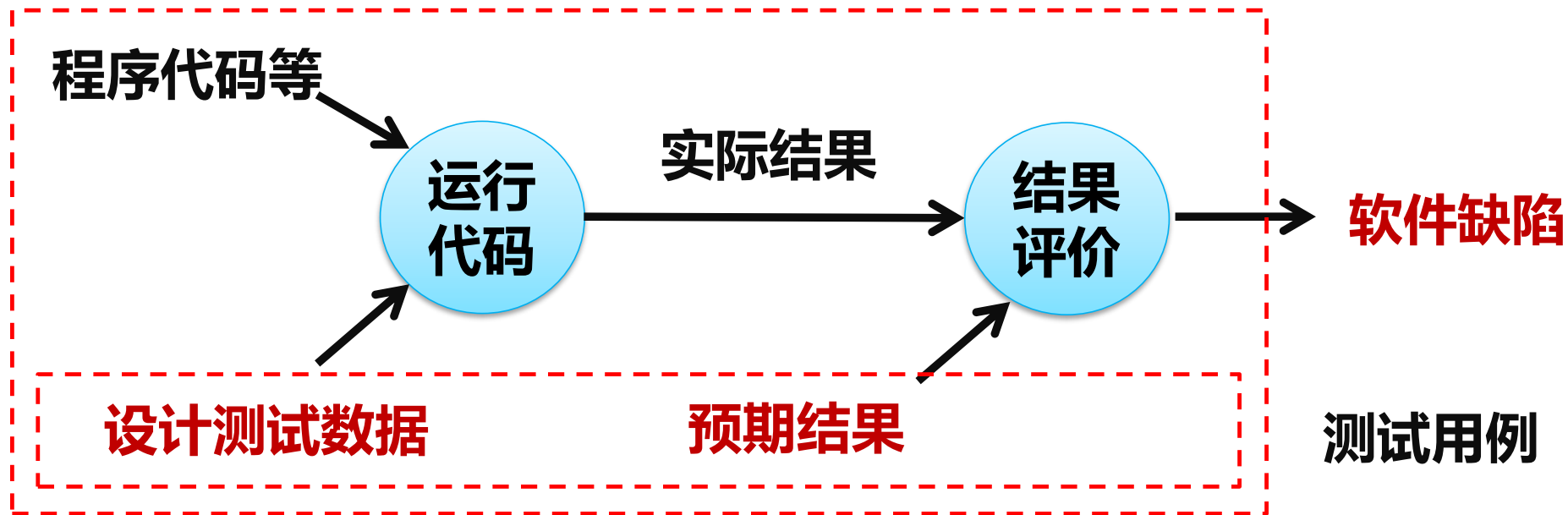


代码审查后要做的工作

- 理解发现和指出的问题
- 修改和更正有问题代码
- 对于无法很快更正的错误，要把错误的信息记录下来，以便适当的时候能够更正

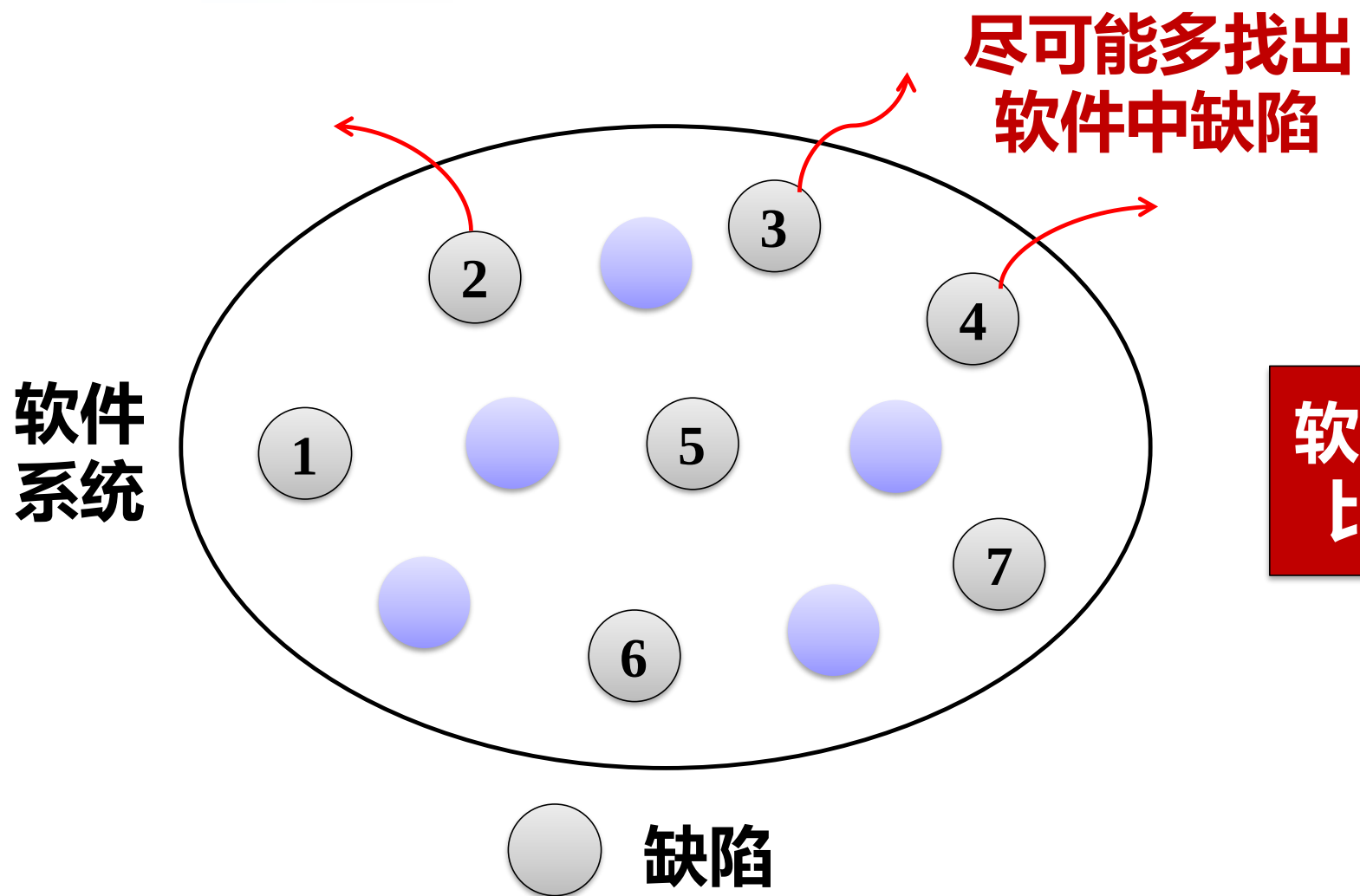
3.4 软件测试技术

- 程序本质上是对数据的处理
- 设计数据(测试用例) → 运行测试用例(程序来处理数据)
→ 判断运行结果(是否符合预期结果)



为软件测试而设计的数据称为测试用例(Test Case)

软件测试的目的和任务



软件缺陷可能隐藏的
比较深，难以发现

软件测试示例

□ 一个加法器程序

- ✓ 功能：给定二个数字，将其相加，然后输出

□ 设计测试数据

- ✓ $\langle 1, 2, 3 \rangle \langle -1, 1, 0 \rangle \langle 0, 0, 0 \rangle$

□ 运行测试程序

- ✓ 输入数据，查看运行结果，判断是否与预期结果一致
- ✓ 如果不一致就意味着有错误

思考和讨论

□ 在你的编程实践中，你是采用何种方式来检查代码质量、发现代码问题的？



内容

1. 程序及质量要求

- ✓ 程序及其内部和外部质量

2. 程序质量保证方法

- ✓ 编码规范、设计方法、代码重用、结对编程

3. 程序质量的分析方法

- ✓ 人工审查、自动化分析、代码测试

4. 编写程序需要解决的问题

- ✓ 需要工程化的方法支持



程序编写面临的挑战!

□程序功能从何而来?

✓谁以及如何来确定软件功能?

□软件规模很大怎么办?

✓500 LOC vs 1M LOC

□软件功能变化如何改写代码?

✓在哪里改、如何改?

□如何保证程序质量?

✓能否想到代码的质量问题, 如何保证



光靠脑子思考能解决程序编写的问题吗?

思考和讨论

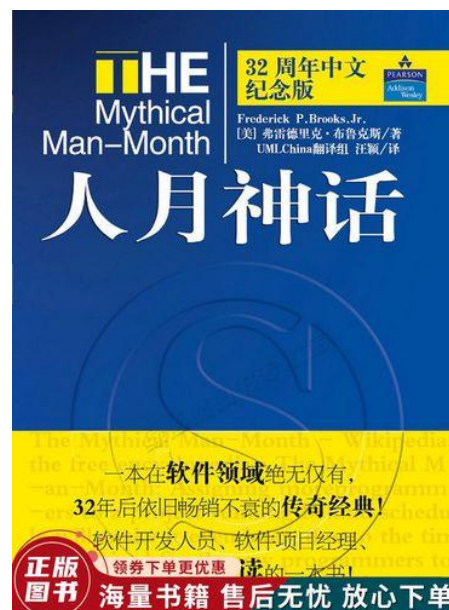
- 如何明确功能、划分模块?
- 如何编写代码、确保质量?
- 对于规模较大应用能行吗?
- 面临什么样的困难和问题?



需要寻找系统化、工程化的方法指导! --- 软件工程

拓展阅读

- **软件科学与工程-学科发展战略**，梅宏等著，高等教育出版社，2021.
- **人月神话**（40周年中文纪念版，Frederick, P.Brooks [美]，清华大学出版社，2015.



本部分知识图谱



小结

□程序的多种**质量要求**

- ✓外在和内在、语法和语义

□确保代码质量的**方法**

- ✓编码规范、设计方法、代码重用、结对编程

□**分析、发现和审查代码**

- ✓人工审查、静态分析、程序测试

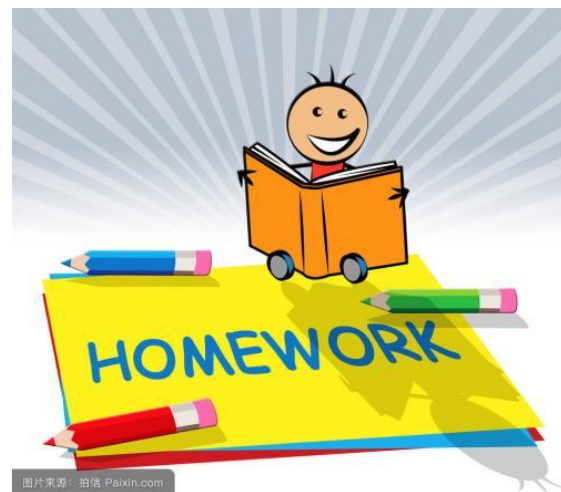
□学会编写**高质量的程序代码**

- ✓需要借助行之有效的、系统化方法的指导

课后的实践任务

任务：提交一个报告，说明程序代码质量实践情况。

- ❑ 开展基于结对的代码分析和维护实践
- ❑ 阅读和掌握编码风格
 - ✓ Java、C/C++编程风格，要求学以致用
- ❑ 了解SonarQube工具
 - ✓ 尝试安装和使用，分析软件的代码质量



Part2: 软件及其特点

1. 软件

✓软件的概念、特点、组成及生命周期

2. 开源软件

✓开源软件实践，托管平台和社区，开源许可证

3. 软件质量

✓软件质量要素及模型

4. 软件特征的变化

✓地位、运行环境、形态、复杂性



建设狗窝和建设大厦的区别

□建设狗窝的特点

- ✓简单
- ✓质量要求不高
- ✓一个人就可完成



**简单、小规模程序
规模在万行代码量**

□建设大厦的特点

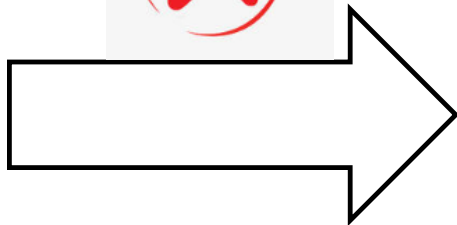
- ✓复杂
- ✓质量高要求很高
- ✓需要团队合作才能完成



**复杂、大规模程序
数十万行以上代码量**

1.1 从程序到软件-工程开发的启发

应用
(如火车票购买、酒店
预订、网上购物)

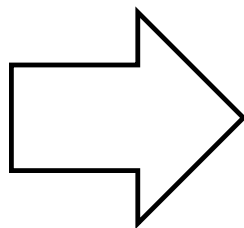


编写程序

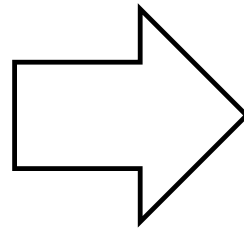
通过一个步骤就将程序写出来不现实



了解需求



设计图纸

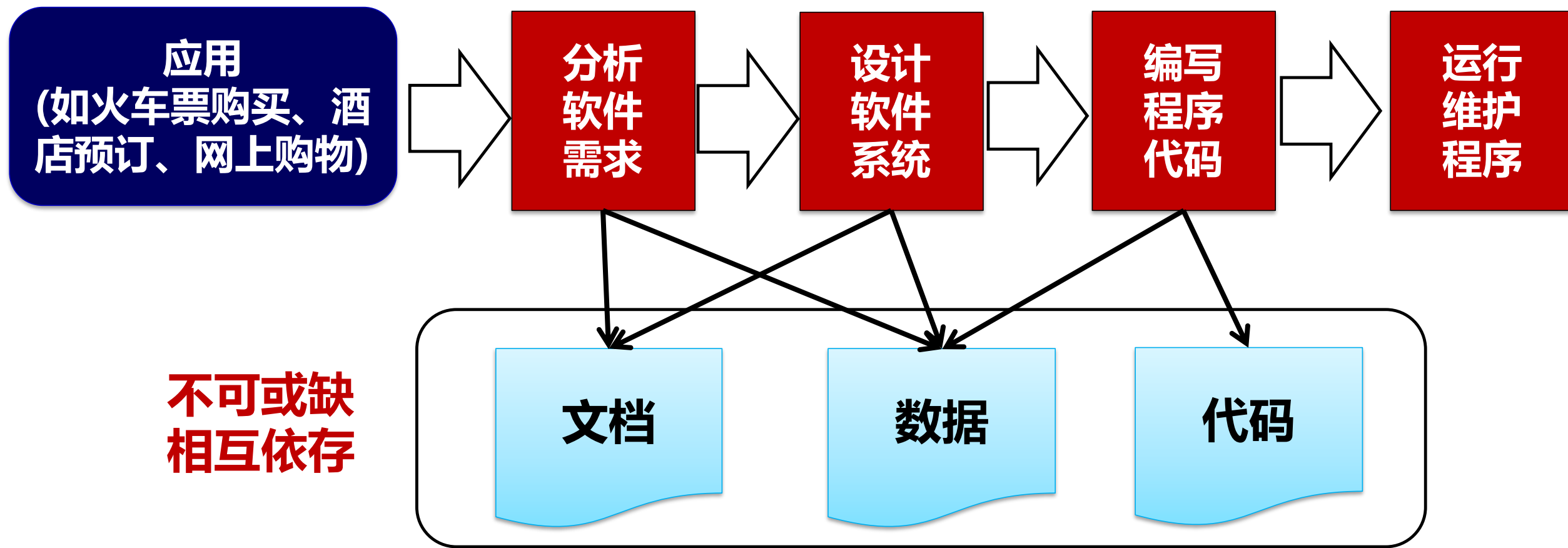


进场施工



采用工程化的方法来解决工程开发的问题

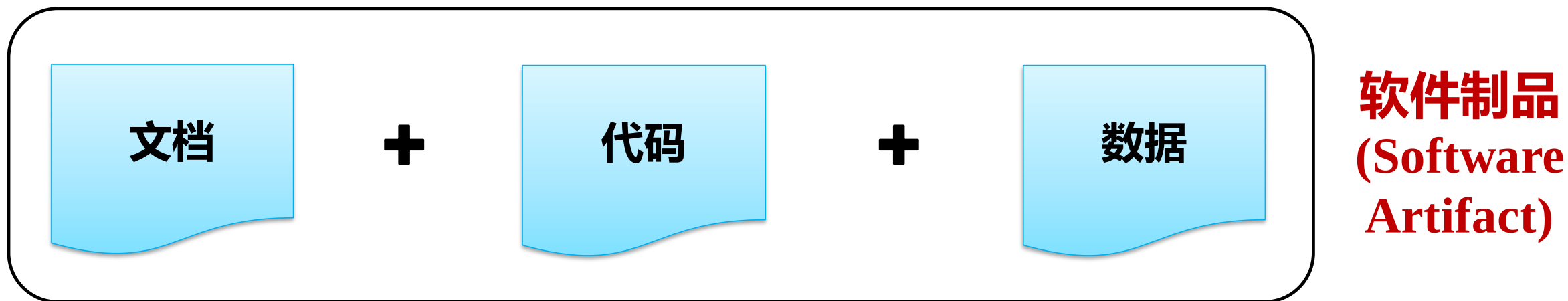
编写出程序需要经历诸多的步骤



要编写出高质量的程序需要循序渐进地开展工作的

1.2 何为软件?

□ 软件(Software)是指在计算机系统的支持下, 能够完成特定功能与性能的程序(Program)、数据(Data)和相关文档(Document)



从开发的角度看软件的构成

何为文档?

□记录软件开发活动和阶段性成果、软件配置及变更的**阐述性资料**

- ✓ 定义和理解软件
- ✓ 记录软件开发成果
- ✓ 辅助不同人员间的交流

□有哪些方面的文档

- ✓ 软件需求文档 - 记录需求
- ✓ 软件设计文档 - 记录设计
- ✓ 软件测试文档 - 记录测试
- ✓ 软件用户手册 - 记录使用
- ✓

“空巢老人看护系统”软件系统的需求构思及描述

阐述性资料

1. 背景介绍

随着人口老龄化问题越来越严重，空巢老人的无人看护将成为一个亟待解决的社会难题。家属外出工作时，经常会惦记独自在家的老人，希望掌握老人的状况（比如：身体状况如何？在做什么？）。但是由于空间距离上的限制，家属不能方便快捷的获取这些信息；很多老年人不会或者不方便使用手机，造成家属与老人之间语音通话上的困难；老年人记忆力普遍衰退，容易忘记服药等一些重要的事情，家属外出时无人监督老人完成这些事情。对于一些患有疾病的老人来说，忘记服药很可能会造成危险；当老人独自在家时，他们可能会遇到摔倒或者中风等突发情况，而外出工作的家属和医疗救护人员不能及时掌握这些危险信息，导致救援不及时。

本软件专为解决空巢老人的看护问题而设计，它可以帮助用户（老人的家属）监控老人在家的状况，帮助老人与家人语音聊天，提醒并监督老人按时服药，并在发生危险时向用户警报或者联系急救部门。

为什么需要文档?

□原因

- ✓ 软件开发牵涉**多方面的工作**，需要记录下来
- ✓ 任何事都记在脑子里，**记不住**，不便于保存
- ✓ 任何事都放在脑子里，**理不清**，不便于分析
- ✓ 任何事都存在脑子里，**讲不明**，不便于交流

- 编码活动的成果记录为**程序代码**
- 其他活动的成果记录为**软件文档**

□编写文档目的

- ✓ **阐述清楚**：内容、逻辑性、条理性
- ✓ **发现问题**：完整、一致、矛盾
- ✓ **开展交流**：便于各种人员的交流
- ✓ **促进管理**：管理软件开发成果



示例：软件设计文档

空巢老人智能看护系统设计文档

1、引言.....	4
1.1 编写目的.....	4
1.2 读者对象.....	4
1.3 软件项目概述.....	4
1.4 文档概述.....	5
1.5 定义.....	5
1.6 参考资料.....	5
2、软件设计约束.....	5
2.1 软件设计目标和原则.....	5
2.2 软件设计的约束和限制.....	6
3、软件设计.....	6
3.1 软件体系结构设计.....	6
3.2 用户界面设计.....	9
3.3 用例设计.....	13
3.4 类设计.....	24
3.5 数据设计.....	32
3.6 部署设计.....	33

何为数据？

□数据是程序的加工处理对象和结果

- ✓需要处理哪些数据
- ✓如何来获得这些数据
- ✓如何来表示这些数据
- ✓如何来存储和检索数据
- ✓如何来传输数据



		
Apple iPhone 11 (A2223) 128GB 白色 移动联通电信4G手机 双卡双 ¥5899.00	华为 HUAWEI Mate 30 5G 麒麟 990 4000万超感光徕卡影像双超级 ¥4499.00	Apple iPhone 11 Pro Max (A2220) 256GB 暗夜绿色 移动联通电信4G ¥10899.00
		
华为 HUAWEI Mate 30 Pro 5G 麒麟 990 OLED环幕屏双4000万徕卡 ¥5899.00	【向往的生活同款】小米10 双模 5G 骁龙865 1亿像素8K电影相机 ¥4349.00	Redmi K30 5G双模 120Hz流速屏 骁龙765G 前置挖孔双摄 索尼6400 ¥1599.00

网上商城中的手机及其数据

数据示例

□数据形式

✓用户、订单、交易、日志
数据

□数据处理

✓表示、获取、存储、检索、
分析

华为 (HUAWEI) 华为Mate20 手机 麒麟980AI智能芯片全面屏超微距影像超大广角 亮黑色 全网通 6GB+64GB

6.53英寸, 4000毫安电池

京 东 价 **¥ 3999.00** 降价通知 累计评价 100+

优 惠 券 **满2000减19** **满1599减15** **满999减9** 更多>>

增值业务 **高价回收, 极速到账** **套 3元1G**

配 送 至 **湖南长沙市长沙县白沙镇** 有货 支持 送运费险 | 一年质保 | 全国联保 | 7天价保

由 亿品汇旗舰店 负责发货, 并提供售后服务.

选择颜色  极光色  亮黑色  宝石蓝  翡冷翠  樱粉金

选择版本 **全网通6GB+64GB** 全网通6GB+128GB

增值保障  2年全保修 ¥199  1年碎屏保修 ¥99  2年京享无忧 ¥249 

白条分期 **不分期** ¥1353起×3期 ¥686.5起×6期 ¥353.25起×12期 ¥186.51起×24期 

1   **加入购物车**

软件运行的本质就是对数据的处理，有些数据是用户提供的，有些数据是软件产生的

思考和讨论：12306软件的数据

□ 哪些数据是由用户提供的？

✓ 以用户注册和购票为例

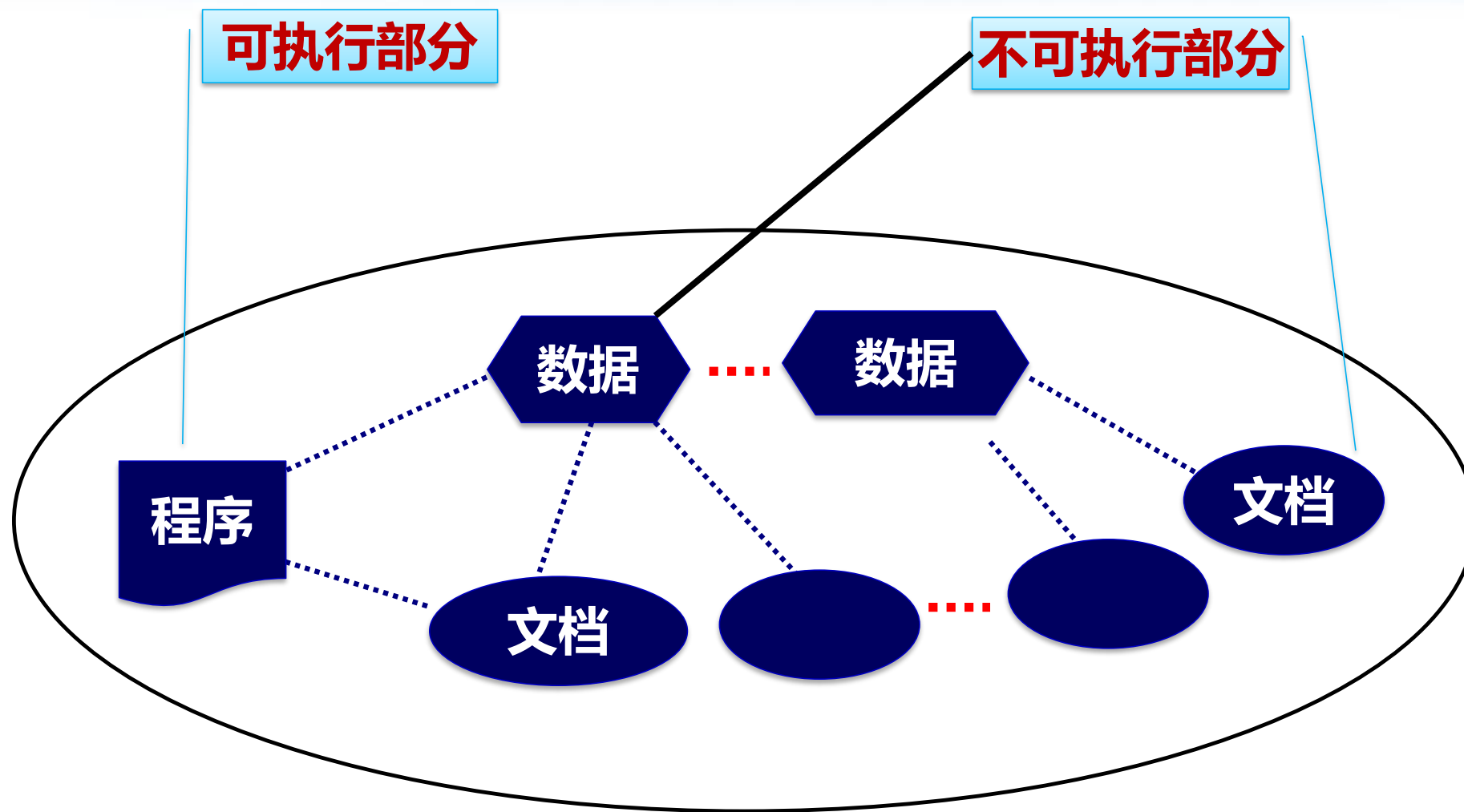
□ 哪些数据是由软件产生的？

✓ 以用户注册和购票为例

□ 没有数据软件会怎么样？



从开发者的角度看软件的构成



构成软件的三类成分作用不同，但都不可或缺

1.3 软件概念给我们的启示

□ 软件 ≠ 程序

- ✓ 软件涉及更多制品
- ✓ 这些制品间有关联、相互依赖

□ 开发软件 ≠ 编写程序

- ✓ 软件开发需要做更多事情
- ✓ 对软件提出更高的要求

软件开发的再认识

□开发具有一定规模和复杂性的软件系统与编写一个简单的程序**不一样**

✓建设简单平房 vs 建设高楼大厦

✓挖一条小沟 vs 建设一条南水北调渠道



何为工程化方法?

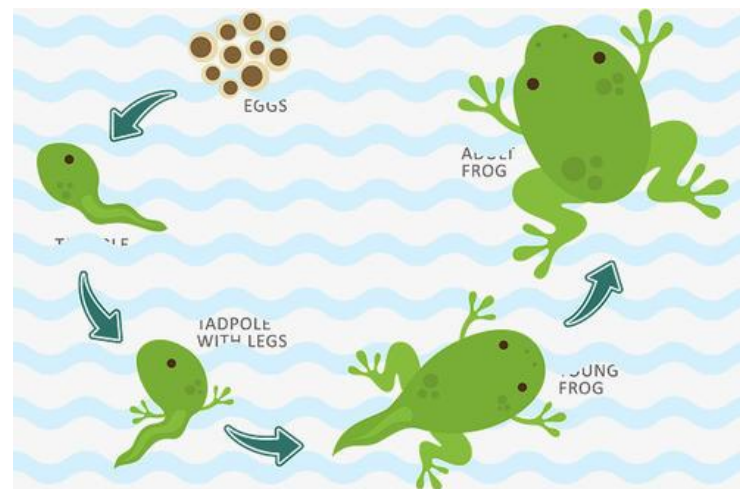
软件的规模和复杂性意味着要采用行之有效的方法——工程方法

开发软件 vs 编写程序

1.4 软件生命周期

□万物均有生命周期(Lifecycle)

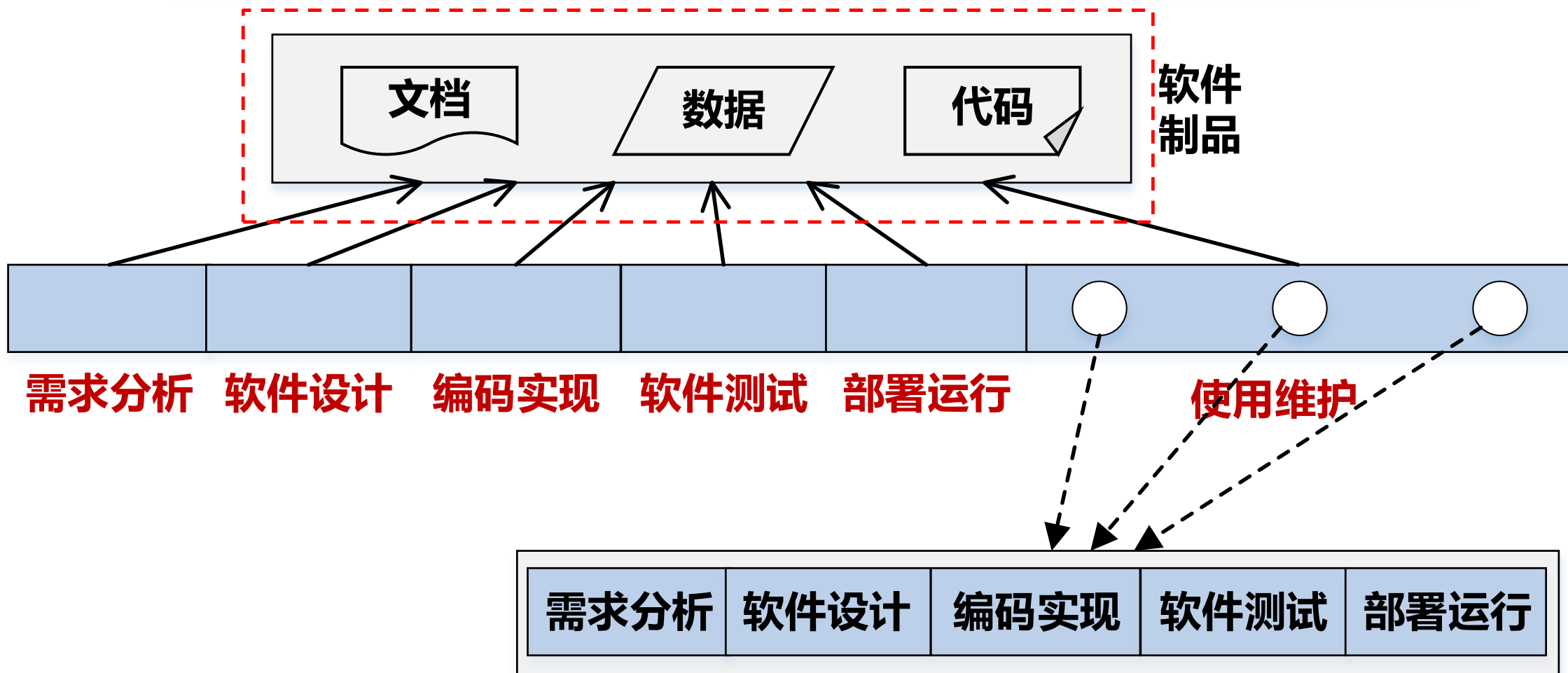
- ✓人：婴儿、幼儿、儿童、少年、...
- ✓树：发芽、育苗、结果、枯萎、死亡、....
- ✓青蛙：受精卵、蝌蚪、去掉尾巴的青蛙 ...



□软件生命周期(Software Lifecycle)

- ✓软件从提出开发开始到最终灭亡所经历的时期
- ✓不同阶段具有不同的特征

软件生命周期(1/2)



从提出开发开始到开发出系统、运行维护以及最终退役的全过程

软件的生命周期(2/2)

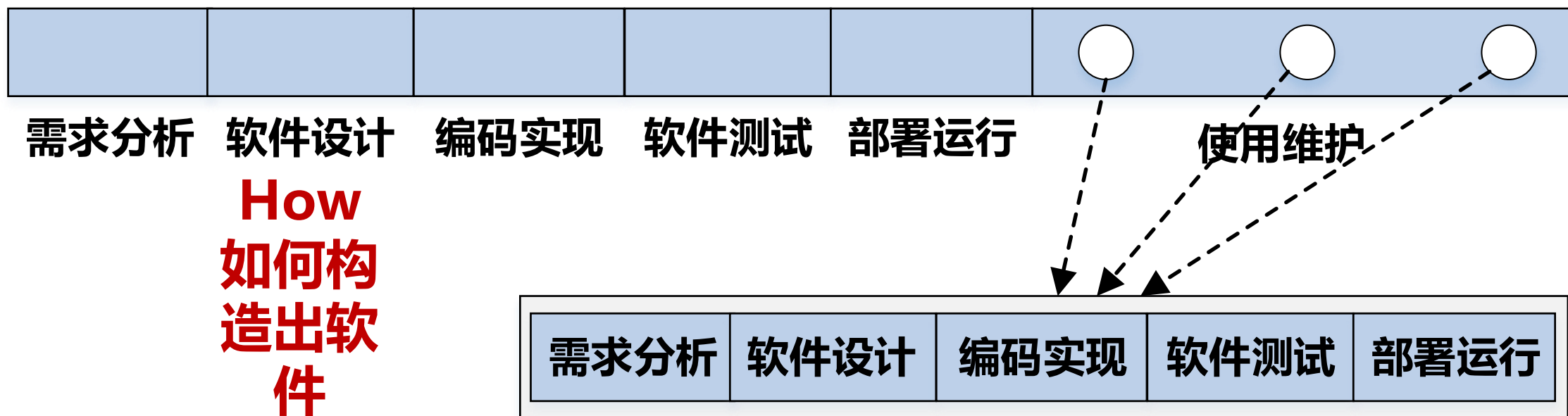
What
软件需求是什么

How
编写程序代码

软件是否缺陷

部署软件运行

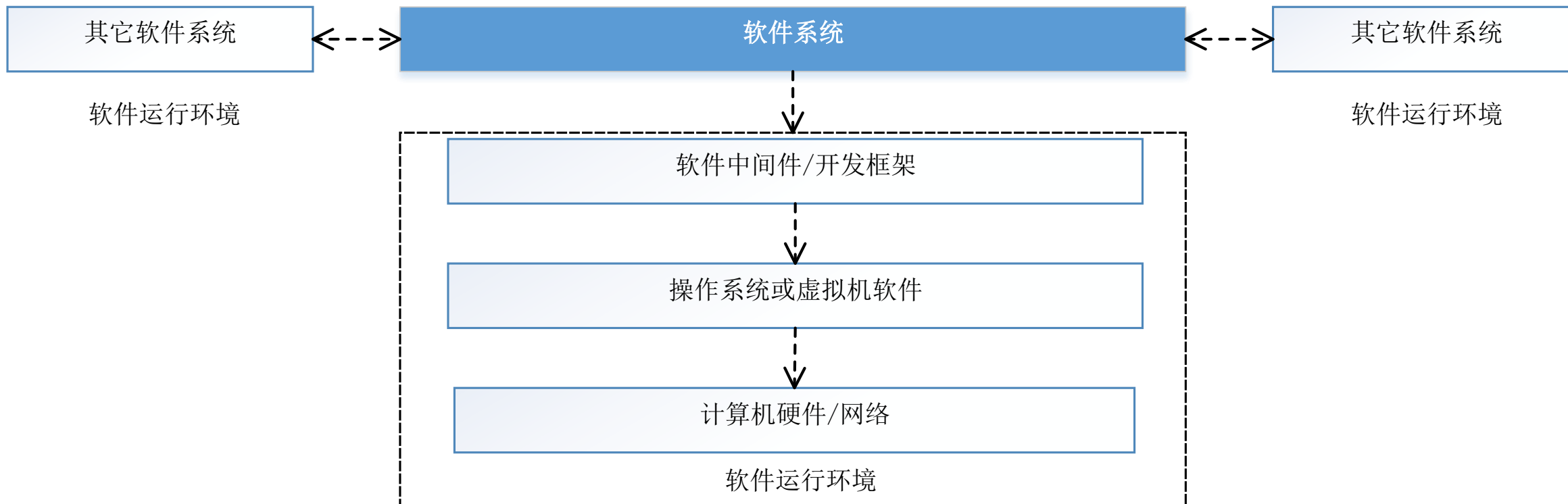
每个阶段会产生不同的软件制品



软件的本质

- 软件的**目的性**：服务于客户或用户，满足他们的要求
- 软件的**组成性**：程序 + 文档 + 数据
- 软件的**系统性**：软件是一类系统，有诸多相关联的要素
- 软件的**驻留性**：依赖于计算系统的软硬件设施来支撑运行

示例：软件的驻留性和系统性



1.5 软件特点

□ 逻辑性

- ✓ **逻辑产品**，思维活动（而非物理活动）的结果、不会磨损和老化

□ 设计开发

- ✓ 是**设计开发**而成的，不是生产制造而成的

□ 易变性

- ✓ 需求**经常变**、难以把控，影响软件的制品即开发过程

□ 复杂性

- ✓ **规模大**：代码行、模块、介入人员、进程、数据等数量非常大
- ✓ **运行复杂**：状态很难追踪和复现

□ 缺陷的隐蔽性

- ✓ 缺陷**隐藏在逻辑代码**中，不像硬件系统那样直观显现，很难被人们所发现和排除

讨论：对比软件和硬件

□对比软件 and 硬件的特点

- ✓表现形式
- ✓开发范式
- ✓需求变化
- ✓复杂性
- ✓系统缺陷



软件系统

VS

硬件系统



1.6 软件分类

□ 应用软件

- ✓ 面向**特定应用领域**的专用软件。它们针对相关行业和领域的特定问题，为其提供基于计算的新颖解决方案。
- ✓ 示例：淘宝、12306、携程、微信、QQ等软件

□ 系统软件

- ✓ 对计算机资源进行管理，**为应用软件的运行提供基础设施和服务**的一类软件。从计算服务的视角，系统软件介于计算机硬件和应用软件之间。
- ✓ 示例：操作系统、数据库管理系统、编译软件、软件中间件等

□ 支撑软件

- ✓ **辅助软件开发和运维**，帮助开发人员完成软件开发和维护工作的一类软件
- ✓ 示例：SonarQube、Visual Studio、Eclipse等

软件的分类

类别	服务对象	软件的功能	发挥的作用
应用软件	行业和领域应用的用户	为特定行业和领域问题解决提供基于软件解决方案，创新应用领域的问题解决模式	提供更为便捷、快速、高效的服务
系统软件	各类应用软件	为应用软件运行和维护提供基础设施和服务，如加载、通讯、互操作、管理等	作为应用软件的运行环境
支撑软件	软件开发者和维护者	为软件系统的开发和维护提供自动和半自动的支持	提高软件开发效率和质量

内容

1. 软件

✓软件的概念、特点、组成及生命周期

2. 开源软件

✓开源软件实践，托管平台和社区，开源许可证

3. 软件质量

✓软件质量要素及模型

4. 当前软件特征的变化

✓地位、运行环境、形态、复杂性



2.1 闭源软件

□何为闭源软件

- ✓软件代码不对用户开放的一类软件，购买软件时只提供可运行软件或服务，没有提供源代码
- ✓以许可证（License）的方式授权用户使用软件

□闭源软件带来的问题

- ✓无法掌握软件内部实现情况（如是否存在安全漏洞和恶意代码），也无法对软件进行修改和完善，极大影响了开发者的创新自由

□示例

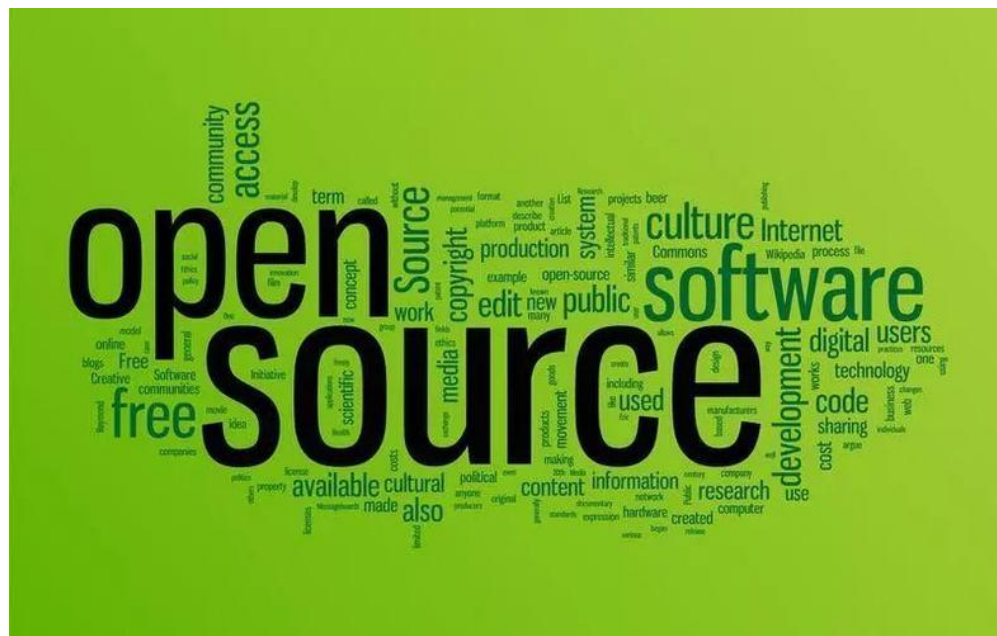
- ✓微软的Windows、Office软件，
- ✓典型企业：微软、IBM、Oracle等



2.2 开源软件

□何为开源软件

- ✓一种源代码可以**自由获取和传播**的计算机软件，其拥有者通过开源许可证赋予被许可人对软件**进行使用、修改和传播**



海量、高质量和有影响力的开源软件

□ 操作系统

✓ Linux、Ubuntu、麒麟、鸿蒙、OpenEuler等

□ 数据库系统

✓ MySQL、PostgreSQL、MongoDB、Redis等

□ 开发平台

✓ Eclipse、Junit、SonarQube、Kubernetes等

□ 人工智能

✓ Tensorflow、Opencv、Caffe、Deeplearning4j等

□ 网络安全

✓ Nmap、curity Onion、Suricata、Bro等

□



你能枚举出其他的开源软件吗？

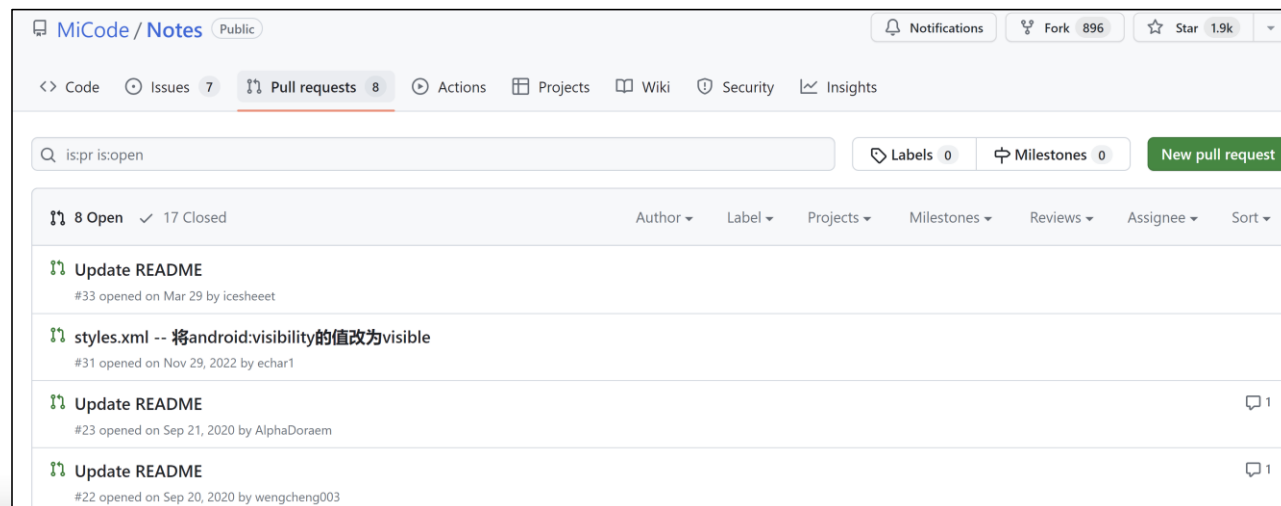
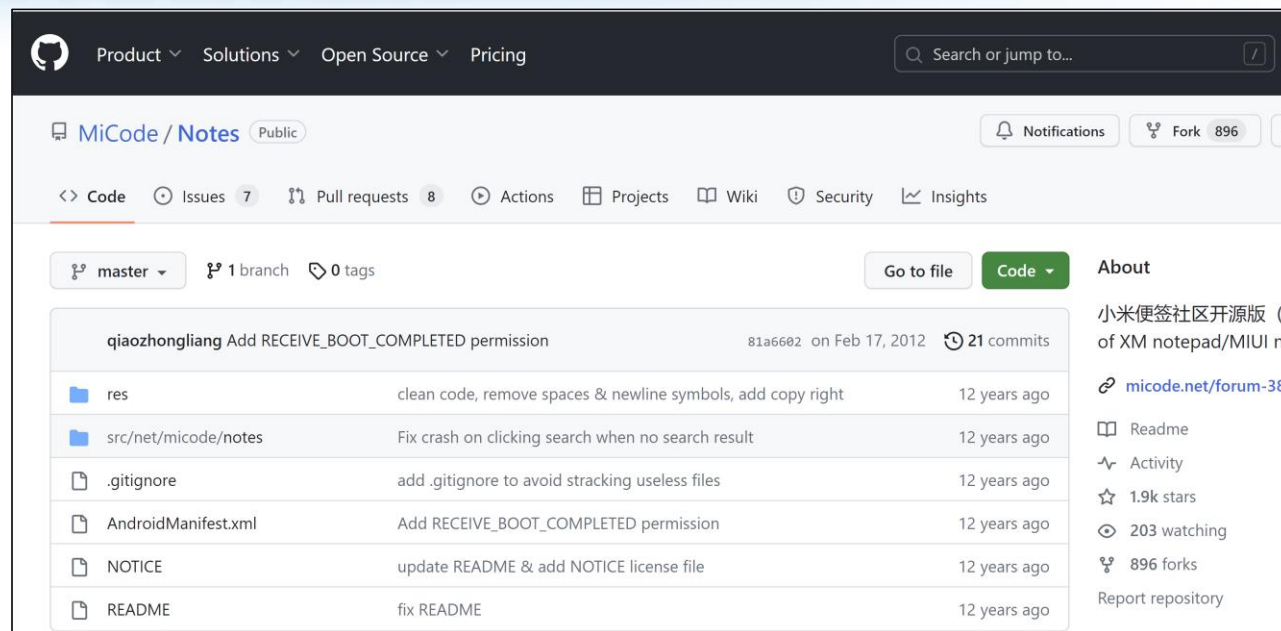
2.3 开源软件托管平台

□ 托管服务

- ✓ 创建软件仓库
- ✓ 支持协同开发
- ✓ 软件质量保证

□ 代表性托管平台

- ✓ Github
- ✓ SourceForge
- ✓ Gitee



开源软件托管平台 - Github

□ Github是全球最有影响力的**开源软件托管平台**

✓ www.github.com, 目前由微软负责经营

The screenshot shows the Github repository page for tensorflow/tensorflow. The top navigation bar includes the Github logo, a search bar, and links for Pull requests, Issues, Marketplace, and Explore. The repository name is tensorflow / tensorflow, with 8.2k Watchers, 155k Stars, and 84.6k Forks. The main content area shows the repository structure with a table of files and folders, including .github, tensorflow, third_party, tools, .bazelrc, .bazelversion, and .gitignore. The right sidebar contains the 'About' section, describing it as an Open Source Machine Learning Framework for Everyone, with links to tensorflow.org, tags like python, machine-learning, deep-neural-networks, deep-learning, neural-network, tensorflow, ml, distributed, and links to the Readme and Apache-2.0 License.

Search or jump to... Pull requests Issues Marketplace Explore

tensorflow / tensorflow Watch 8.2k Star 155k Fork 84.6k

<> Code Issues 3.8k Pull requests 187 Actions Projects 1 Security 35 Insights

master 45 branches 136 tags Go to file Add file Code

tensorflow-gardener Integrate LLVM at llvm/llvm-project@ad12590 23920c8 3 minutes ago 109,746 commits

.github	PR #48687: Update bot_config.yml	8 days ago
tensorflow	Merge pull request #47897 from lgeiger:cleanup-compat-v2	12 minutes ago
third_party	Integrate LLVM at llvm/llvm-project@ad12590	3 minutes ago
tools	Merge pull request #25673 from Ryan-Qiyu-Jiang:env_capture_script_m...	2 years ago
.bazelrc	Enable --remote_download_minimal for Windows RBE build	3 days ago
.bazelversion	Updating TensorFlow's bazel version 4.0.0 (the first LTS release).	3 months ago
.gitignore	Ignore CoreML BUILD files which are generated by the configure script	13 months ago

About

An Open Source Machine Learning Framework for Everyone

tensorflow.org

python machine-learning deep-neural-networks deep-learning neural-network tensorflow ml distributed

Readme

Apache-2.0 License

开源软件托管平台 - SourceForge

□ 一个有影响力的**开源软件开发平台和仓库**，为开源软件提供**存储、协作和发布的支持**

✓ sourceforge.net



开源软件托管平台 - Gitee

□中国的开源软件托管平台

✓ 汇聚了国内众多的开发者和主要软件企业的开源软件项目，如华为OpenHarmony操作系统、百度的PaddlePaddle等

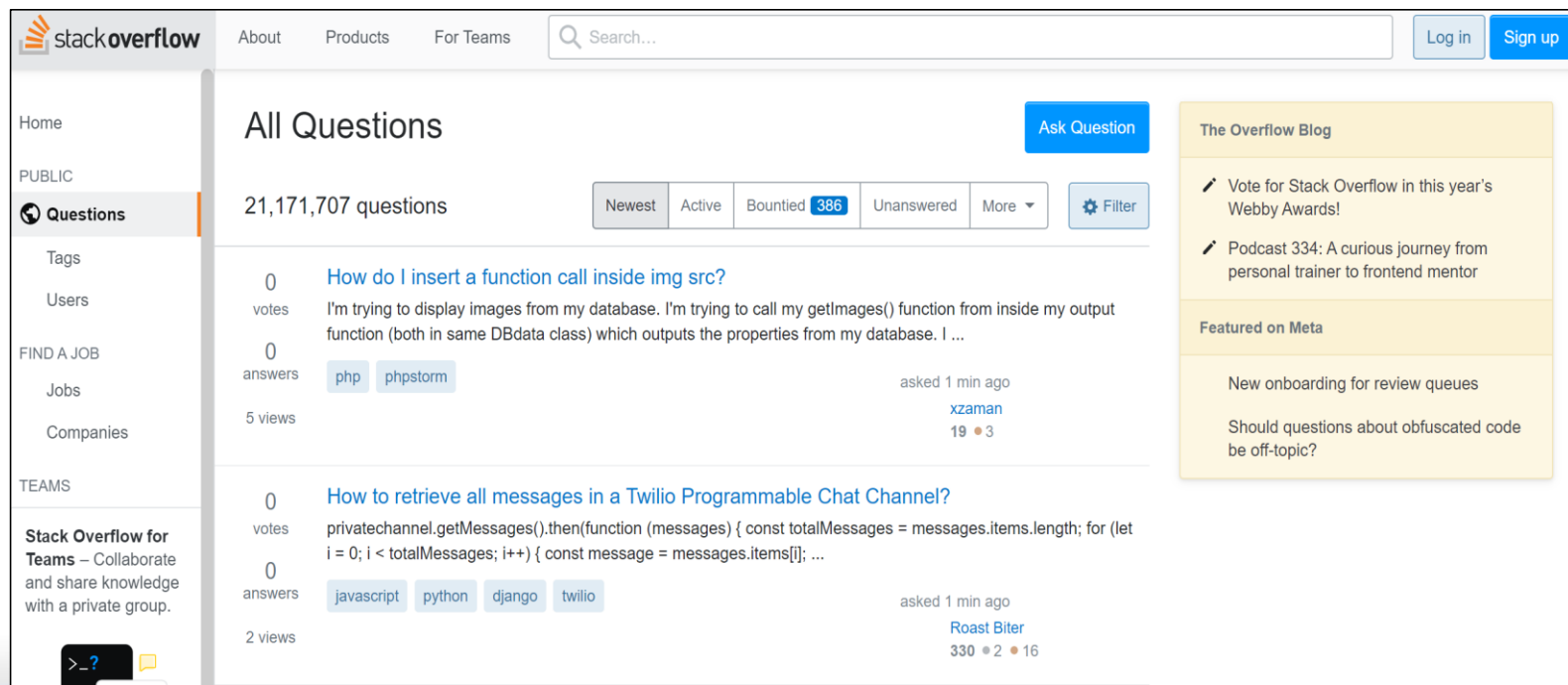
✓ www.gitee.com

The image is a screenshot of the Gitee website's homepage. At the top, there is a navigation bar with the Gitee logo, links for '开源软件' (Open Source Software), '企业版' (Enterprise Edition) with a '特惠' (Special Offer) tag, '高校版' (University Edition), and '博客' (Blog). A search bar labeled '搜开源' (Search Open Source) is on the right, along with '登录' (Login) and '注册' (Register) buttons. The main content area features the title '云端软件开发协作平台' (Cloud Software Development Collaboration Platform) and the tagline '帮助开发者/团队/企业更好地管理代码，让软件研发更高效' (Help developers/teams/enterprises better manage code, making software development more efficient). Below this are two buttons: '注册 Gitee' (Register Gitee) and '免费开通企业版' (Free Enterprise Edition). A statistics table follows, showing the number of developers, code repositories, enterprise customers, and universities. To the right of the table is a large illustration of a person in a white shirt and red pants pointing at a large screen displaying various charts and data.

软件开发知识分享社区 - Stack Overflow

❑ 全球最大的**软件开发知识分享平台**，它支持开发者交流和讨论软件开发问题、分享软件开发经验和知识，进而促进相关开发问题的解决，推动开源软件的开发

✓ stackoverflow.com



软件开发知识分享社区 - CSDN

□中国的软件开发知识分享社区

✓技术博客、学习社区、代码托管等

✓www.csdn.net



2.4 开源软件的优势

□采购和开发的成本更低

- ✓开源软件通常是免费的，即使要付费，其费用也非常低廉

□软件质量更高、更安全

- ✓核心代码都在公众的视野之中，代码问题（如缺陷、安全漏洞等）很容易被人发现

□软件研制和交付的更快

- ✓基于开源软件的项目开发可以更为快速地给用户交付软件产品

□软件功能更为强大

- ✓大量的软件开发者不仅参与软件开发，贡献他们的代码，而且还参与软件的创新，提出和构思软件需求，不断完善软件功能

为什么开源软件具有这些优势？

内容

1. 软件

✓软件的概念、特点、组成及生命周期

2. 开源软件

✓开源软件实践，托管平台和社区，开源许可证

3. 软件质量

✓软件质量要素及模型

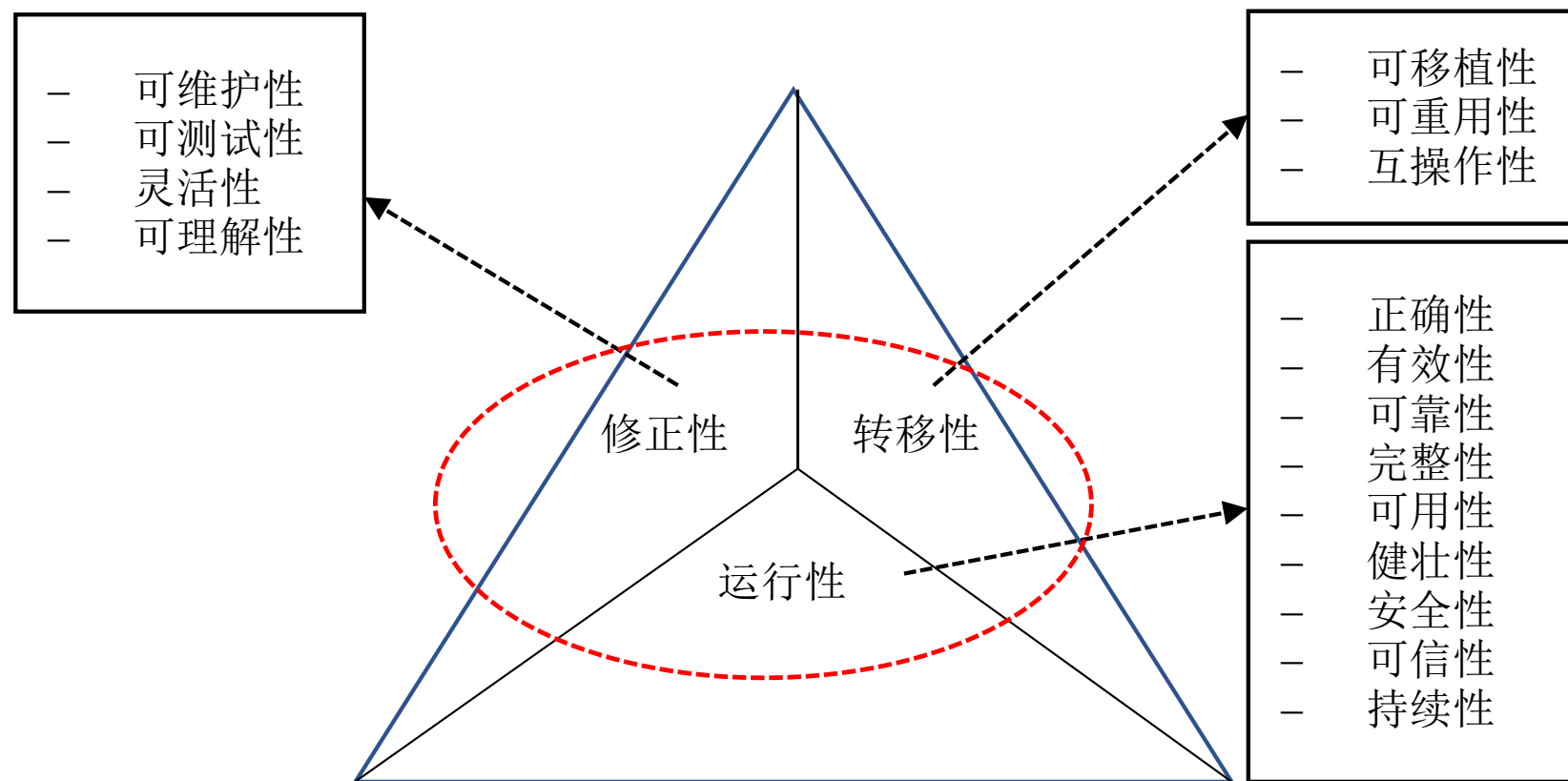
4. 当前软件特征的变化

✓地位、运行环境、形态、复杂性



3.1 软件质量

□软件质量是指软件**满足给定需求的程度**，它是软件的生命线



软件质量同样可分为内部质量和外部质量

3.2 软件质量要素 (1/3)

□正确性 (Correctness)

- ✓软件满足规格说明和用户要求的程度，即在预定环境下能正确地完成预期功能和非功能需求的程度。

□可靠性 (Reliability)

- ✓在规定条件下、限定时间范围内，软件系统完成预期功能、不引起系统故障的能力。

□健壮性 (Robustness)

- ✓在计算环境发生故障、输入无效数据或操作错误等意外情况下，软件仍能做出适当响应的程度。

□有效性 (Efficiency)

- ✓软件利用计算资源和存储资源以实现其功能的能力。

软件质量要素 (2/3)

□安全性

- ✓包括系统安全 (Safety) 和信息安全 (Security) , 系统安全是指软件能及时有效地避免给人员、设施、环境、经济等造成损害, 信息安全是指软件能有效防控各类的非法获取、传播和使用

□可维护性 (Maintainability)

- ✓是否易于对其进行修改以更正错误、增强功能、适应新运行环境

□可移植性 (Portability)

- ✓把软件从一种运行环境转移到另一种运行环境下运行的难易程度

□可重用性 (Reusability)

- ✓软件模块、构件、设计方案等在其他软件开发中被再次使用程度

软件质量要素 (3/3)

□可理解性 (Comprehensibility)

- ✓ 软件开发者或用户理解该软件系统的容易程度

□可信性 (Dependability)

- ✓ 采取有效措施确认软件满足人们的要求和期望

□持续性 (Sustainability)

- ✓ 面对各种突发异常事件，仍能提供令人满意的服务的能力

□可用性 (Usability)

- ✓ 使用和操作软件系统难易程度

□互操作性 (Interoperability)

- ✓ 软件系统与其他系统进行交换信息、协同工作的能力

2019年华为公司发布的第一号文件

□ “全面提升软件工程能力与实践，打造可信的高质量产品”

✓把可信作为第一优先级，放在功能、特性和进度之上

□可信软件的基本要求

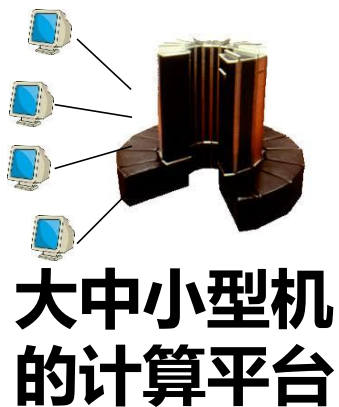
- ✓**安全性**以抗击攻击，保护业务和数据的机密性、完整性和可用性
- ✓**韧性**（Resilience）以确保系统受攻击时仍能保持良好的运行状态，具有快速恢复的能力
- ✓**隐私性**（Privacy）以保护用户的数据及其隐私
- ✓**可靠性**和**可用性**以确保软件产品长期无故障运行，具备快速恢复和自我管理的能力，提供可预期的、一致的服务。

4.1 软件的地位和作用

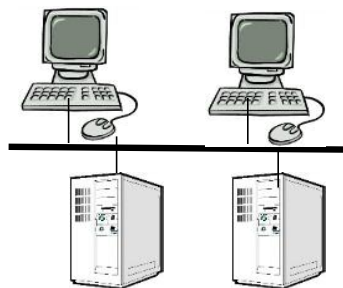
- 深入到社会、经济、生活的方方面面，**无处不在**
- 作为一种**创新的工具**，深刻地改变各个行业和领域，创新问题解决模式和方式，影响人们日常学习、生活和工作
 - ✓ “12306” 改变了购票方式，淘宝和京东改变了购物方式
- 软件是实现行业和领域进行信息化融合和改造，实现创新性发展的**使能技术和重要利器**
 - ✓ 企业信息化改造，图书借阅等
- 软件已经成为人类社会的**关键性基础设施**
 - ✓ 就像电力、交通设施一样，发挥基础性的作用

4.2 软件运行环境的变化和发展

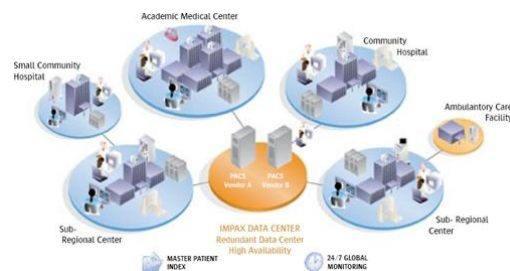
从前端的PC终端、可穿戴设备、智能手机到后端的云中心、高性能计算中心



基于PC的
计算平台



基于局域网的
分布计算平台



基于互联网的
计算平台



无处不在的
计算

从孤立、独立、局域和可控的计算环境到分布、开放、动态、难控、无处不在计算环境

4.3 软件形态

□不是单一系统，而是**系统之系统**

✓诸多系统联盟和组合而成

□不仅是社会技术系统，还是一种**生态系统**

✓社会、信息、物理等要素共存

□不是同构系统，而是异构、多样的**系统联盟**

✓要素异构，客观存在，也是必然

□不在封闭环境，而在**开放环境中**

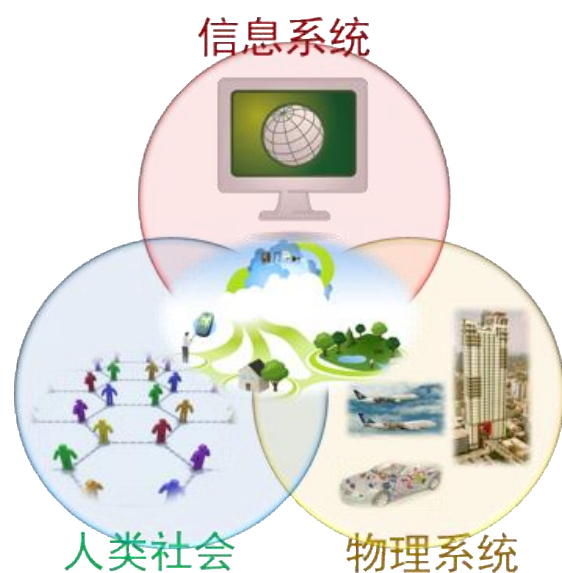
✓环境和系统相互作用，环境开放如互联网、战场环境

□不是封闭系统，而是**动态适应系统**

✓要素、关系、联盟等持续变化，边界不明确

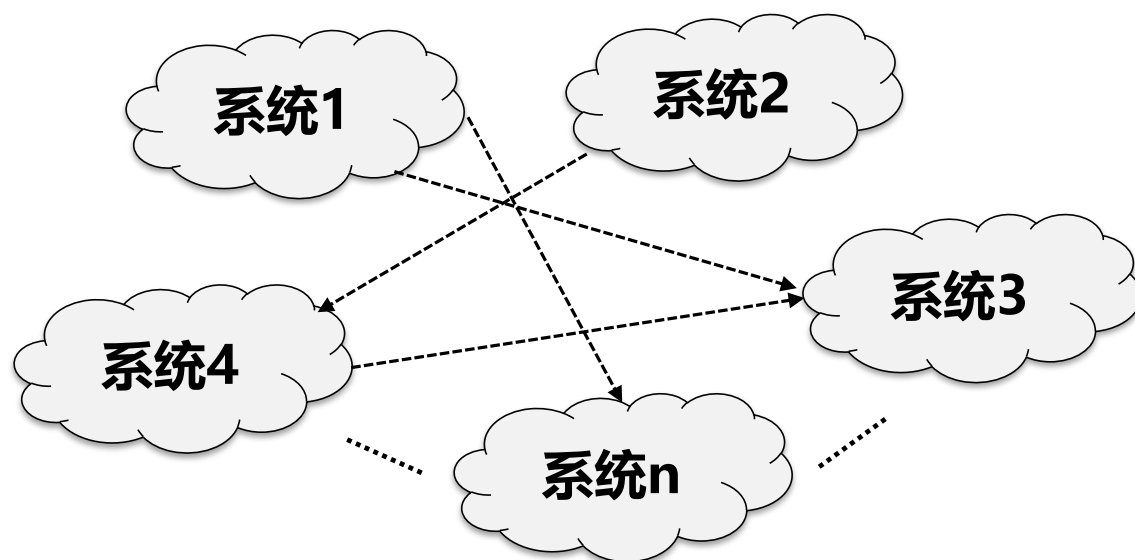
软件形态-社会技术系统

- 由人、社会组织、物理设备、过程等要素共同组成和相互作用的人机物共生系统
- 软件系统无法单独存在，需与物理、社会系统交互作用



软件形态-系统之系统

- 由一组面向任务、服务于不同对象的**子系统**构成
- 每个子系统可**独立运作**并能提供相对独立功能
- 整个系统通过各独立系统间**交互**来实现全局任务



软件形态-分布式异构系统

- 拥有大量形式多样、**地理或者逻辑上是分布的，分散部署在互联网上的软件实体**
- 软件实体的分布性是**必须**的，因为越来越多的应用本身就是分布的，软件实体的分布性有助于提高软件系统的可靠性和安全性
- 构成软件系统的软件实体通常是异构的，异构性是一种**必然**

软件形态-动态演化系统

□系统的边界和需求的**不确定性和持续演变性**

□**动态开放的特点**

✓软件系统需要根据外部环境的变化而不断地调整自身，包括系统的体系结构和交互协作等等，进而表现出持续演化的特点

□系统的运维和系统的运行需要交织在一起

- 城市交通管理系统
- 银行服务信息系统
- 医疗保证信息系统
- 作战指挥控制系统
-

□大规模复杂信息技术系统是由大量**相对独立、自我控制和管理**的系统**组装**而成一系统联盟

- ✓分属不同的组织，并由他们来管理
- ✓共同工作和相互共存
- ✓并非一开始就设计好，而是演化而成
- ✓联盟动态变化

开发系统联盟需要采用社会技术观点，借助系统工程的方法

DOI:10.1145/2209249.2209289

The reductionism behind today's software-engineering methods breaks down in the face of systems complexity.

BY IAN SOMMERVILLE, DAVE CLIFF, RADU CALINESCU, JUSTIN KEEN, TIM KELLY, MARTA KWIATKOWSKA, JOHN MCDERMID, AND RICHARD PAIGE

Large-Scale Complex IT Systems

ON THE AFTERNOON of May 6, 2010, the U.S. equity markets experienced an extraordinary upheaval. Over approximately 10 minutes, the Dow Jones Industrial Average dropped more than 600 points, representing the disappearance of approximately \$800 billion of market value. The share price of several blue-chip

multinational companies fluctuated dramatically; shares that had been at tens of dollars plummeted to a penny in some cases and rocketed to values over \$100,000 per share in others. As suddenly as this market downturn oc-

curred, it reversed, so over the next few minutes most of the loss was recovered and share prices returned to levels close to what they had been before the crash.

This event came to be known as the "Flash Crash," and, in the inquiry report published six months later,¹ the trigger event was identified as a single block sale of \$4.1 billion of futures contracts executed with uncommon urgency on behalf of a fund-management company. That sale began a complex pattern of interactions between the high-frequency algorithmic trading systems (algorithms) that buy and sell blocks of financial instruments on incredibly short timescales.

A software bug did not cause the Flash Crash; rather, the interactions of independently managed software systems created conditions unforeseen

» key insights

- Conditions of systems, in which the system elements are managed and owned independently, pose challenging new problems for systems engineering.
- When the fundamental basis of engineering—reductionism—breaks down, incremental improvements to current engineering techniques are unable to address the challenges of developing, integrating, and deploying large-scale complex IT systems.
- Developing complex systems requires a socio-technical perspective involving human, organizational, social, and political factors, as well as technical factors.

软件形态-生态系统

□“运行”生态

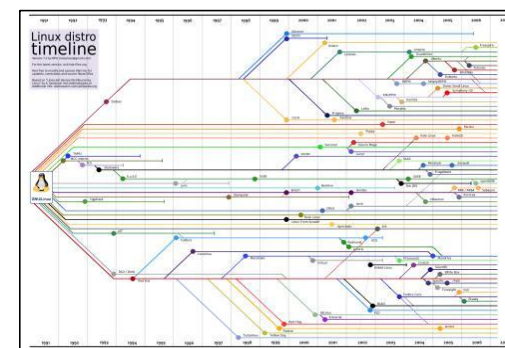
- ✓软件的运行需要依靠“生态”的支撑
- ✓从计算和物理设备、社会系统到OS和运行环境

□“演化”生态

- ✓软件的演化基于特定的“生态”支撑
- ✓从最初的“源头”到最新“版本”

□“人员”生态

- ✓涉及到大量、开放的介入人员
- ✓使用者、开发者、运维者等



生态：共同环境 + 诸多要素 + 独立演化 + 相互依存

4.4 软件系统的规模

□规模的表现及变化

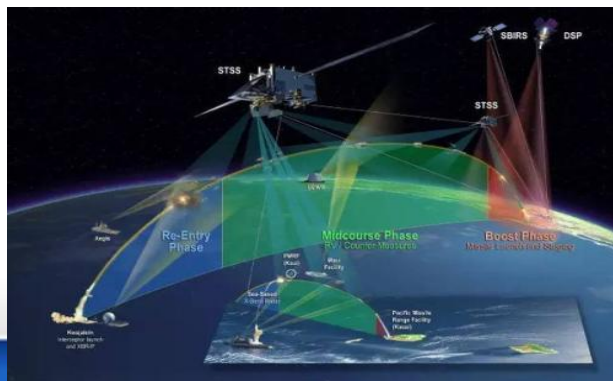
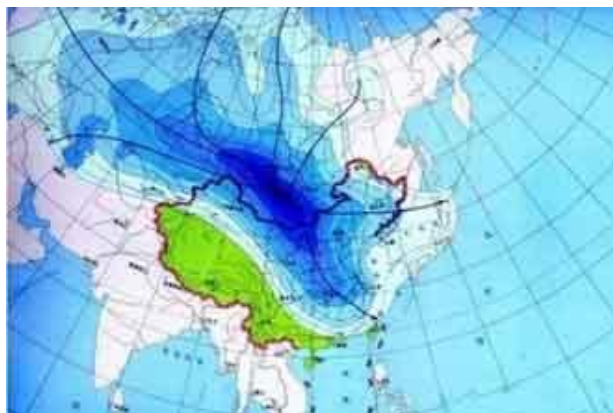
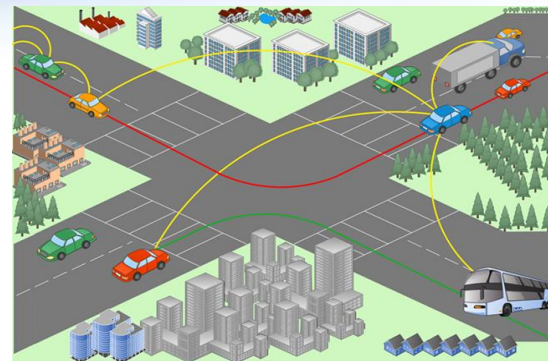
- ✓构成软件系统的代码行数量、软件运行时的进程和线程以及它们之间的交互数量、软件需处理的数据量、软件连接的各类设备和人员**数量等不断增加**

□软件规模示例

- ✓Windows XP有3500万行代码量，Windows Vista有5000万行代码量，Windows 7的代码量大约是7000万行
- ✓宝马7系的软件总量超过了2亿行，特斯拉S的内嵌软件代码行总量超过4亿行，空客A380中的软件代码行总量超过了10亿行
- ✓F-35中80%左右的系统功能由软件来实现，软件代码量约为800万行

软件应用的拓展及深化

- 高性能计算
- 信息物理系统
- 智能机器人
- 云计算
- 健康医疗
- 城市交通
- 军事信息系统
- 航空航天



习题

1-3、 1-4、 1-5、 1-13、 1-18、 1-22